Plano de Estudos: Domine o Desenvolvimento Web em 30 Dias

Este plano foi cuidadosamente elaborado para o guiar na aprendizagem de **HTML**, **CSS** e **JavaScript** em 30 dias, com o objetivo de o capacitar a criar os seus próprios websites. Cada dia inclui uma explicação detalhada do tópico, exemplos de código práticos e um exercício para consolidar o conhecimento.

Semana 1: Fundamentos de HTML (A Estrutura da Web)

Dia 1: Introdução à Web e HTML Básico

Conceitos

O **HTML** (HyperText Markup Language) é a linguagem de marcação padrão para criar páginas web. Ele descreve a estrutura de uma página web semanticamente e consiste numa série de elementos. Os elementos HTML dizem ao navegador como exibir o conteúdo.

Uma página web é um documento que pode ser exibido num navegador web, como o Google Chrome, Firefox ou Safari. A World Wide Web (WWW) é um sistema de documentos interligados e outros recursos da web, acedidos através da Internet.

Estrutura Básica de um Documento HTML

Todo documento HTML começa com uma declaração `<!DOCTYPE html>` e a tag `<html>`. Dentro de `<html>`, temos duas secções principais: `<head>` e `<body>`.

- A secção `<head>` contém metadados sobre a página (informações que não são exibidas diretamente no navegador, mas são importantes para ele e para os motores de busca), como o título da página, links para folhas de estilo CSS e scripts JavaScript.
- A secção `<body>` contém todo o conteúdo visível da página web, como textos, imagens, links, tabelas, etc.

Exemplo de Código: `index.html`

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Minha Primeira Página Web</title>
</head>
<body>
    <h1>Bem-vindo ao Meu Site!</h1>
    <p>Esta é a minha primeira página web. Estou a aprender HTML.</p>
    <p>Aqui está uma lista de coisas que gosto:</p>
    <ul>
        <li>Programação</li>
        <li>Café</li>
        <li>Música</li>
    </ul>
    <p>E aqui está uma lista numerada:</p>
    <ol>
        <li>Primeiro item</li>
        <li>Segundo item</li>
        <li>Terceiro item</li>
    </ol>
</body>
</html>
```

Explicação do Exemplo

- `<!DOCTYPE html>` : Declara o tipo de documento como HTML5.
- `<html lang="pt">` : O elemento raiz de uma página HTML. O atributo `lang="pt"` define o idioma da página como português, o que é útil para motores de busca e leitores de ecrã.
- `<head>` : Contém metadados.
  - `<meta charset="UTF-8">` : Define a codificação de caracteres para UTF-8, permitindo a exibição correta de caracteres especiais.
  - `<meta name="viewport" content="width=device-width, initial-scale=1.0">` : Configura a viewport para tornar a página responsiva em diferentes dispositivos.
  - `<title>Minha Primeira Página Web</title>` : Define o título que aparece na aba do navegador.
- `<body>` : Contém o conteúdo visível.
  - `<h1>` : Um cabeçalho de nível 1 (o mais importante).
  - `<p>` : Um parágrafo de texto.
  - `<ul>` : Uma lista não ordenada (unordered list).
  - `<li>` : Um item de lista (list item).
  - `<ol>` : Uma lista ordenada (ordered list).

Exercício do Dia 1

Crie um novo arquivo HTML chamado `meu_perfil.html`. Neste arquivo, inclua:

1. Um título principal (`<h1>`) com o seu nome.
2. Um parágrafo (`<p>`) a descrever-se brevemente.
3. Uma lista não ordenada (`<ul>`) com 3 dos seus hobbies favoritos.
4. Uma lista ordenada (`<ol>`) com 3 objetivos que tem para aprender desenvolvimento web.

Salve o arquivo e abra-o no seu navegador para ver o resultado.

Dia 2: Atributos, Links e Imagens

Conceitos

Os **atributos** fornecem informações adicionais sobre os elementos HTML. Eles são sempre especificados na tag de abertura e geralmente vêm em pares nome/valor, como `name="value"`.

Links são a espinha dorsal da web, permitindo a navegação entre páginas. São criados com a tag `<a>` (âncora) e o atributo `href` (Hypertext REFerence) especifica o URL de destino.

Imagens são inseridas usando a tag `<img>`. Esta é uma tag vazia (não tem tag de fecho) e requer os atributos `src` (source, caminho da imagem) e `alt` (alternative text, texto alternativo para acessibilidade).

Exemplo de Código: `index.html` (continuação)

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Minha Primeira Página Web</title>
</head>
<body>
    <h1>Bem-vindo ao Meu Site!</h1>
    <p>Esta é a minha primeira página web. Estou a aprender HTML.</p>

    <h2>Links Úteis</h2>
    <ul>
        <li><a href="https://www.w3schools.com/html/" target="_blank">W3Schools
        <li><a href="https://developer.mozilla.org/pt-BR/docs/Web/HTML" target=
        <li><a href="meu_perfil.html">Ver Meu Perfil (página local )</a></li>
```

```
    </ul>

    <h2>Minha Imagem Favorita</h2>
    <img src="https://via.placeholder.com/300x200" alt="Exemplo de imagem place
    <p>Esta é uma imagem de exemplo. O atributo `alt` é importante para acessib
</body>
</html>
```

Explicação do Exemplo

- `<a href="URL" target="_blank">Texto do Link</a>` :
  - `href` : O destino do link. Pode ser um URL externo ou um caminho para outra página no seu próprio projeto.
  - `target="_blank"` : Abre o link numa nova aba do navegador.
- `<img src="URL_DA_IMAGEM" alt="Texto Alternativo" width="largura" height="altura">` :
  - `src` : O caminho para o arquivo da imagem. Pode ser um URL ou um caminho local (ex: `imagens/minha_foto.jpg`).
  - `alt` : Texto descritivo da imagem, crucial para acessibilidade (leitores de ecrã) e quando a imagem não pode ser carregada.
  - `width` e `height` : Definem as dimensões da imagem. É uma boa prática incluí-los para evitar

    problemas de layout enquanto a imagem carrega.

Exercício do Dia 2

Edite o arquivo `meu_perfil.html` que criou no Dia 1:

1. Adicione um link para o seu perfil do LinkedIn (ou outra rede social) na secção de descrição pessoal. Certifique-se de que o link abre numa nova aba.
2. Insira uma imagem sua (ou uma imagem placeholder) na página. Use os atributos `src` , `alt` , `width` e `height` .
3. Crie um link para a página `index.html` (que criaremos no futuro, mas pode ser um link "morto" por agora) no final da página `meu_perfil.html` .

Dia 3: Formulários e Tabelas

Conceitos

Formulários são essenciais para interagir com o utilizador, permitindo a recolha de dados (nomes, emails, mensagens, etc.). O elemento principal é `<form>` , e dentro dele, usamos

vários tipos de elementos de input como `<input>`, `<textarea>`, `<select>`, `<button>`.

Atributos importantes do `<form>` incluem `action` (para onde os dados são enviados) e `method` (como os dados são enviados, geralmente `GET` ou `POST`).

Tabelas são usadas para exibir dados tabulares de forma estruturada. Os elementos principais são `<table>`, `<thead>` (cabeçalho da tabela), `<tbody>` (corpo da tabela), `<tr>` (linha da tabela), `<th>` (cabeçalho da célula) e `<td>` (célula de dados).

Exemplo de Código: `index.html` (adicionar formulário e tabela)

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Minha Primeira Página Web</title>
</head>
<body>
    <h1>Bem-vindo ao Meu Site!</h1>
    <p>Esta é a minha primeira página web. Estou a aprender HTML.</p>

    <h2>Links Úteis</h2>
    <ul>
        <li><a href="https://www.w3schools.com/html/" target="_blank">W3Schools
        <li><a href="https://developer.mozilla.org/pt-BR/docs/Web/HTML" target=
        <li><a href="meu_perfil.html">Ver Meu Perfil (página local )</a></li>
    </ul>

    <h2>Minha Imagem Favorita</h2>
    <img src="https://via.placeholder.com/300x200" alt="Exemplo de imagem place
    <p>Esta é uma imagem de exemplo. O atributo `alt` é importante para acessib

    <h2>Formulário de Contacto</h2>
    <form action="/submit-form" method="post">
        <label for="nome">Nome:</label>

        <input type="text" id="nome" name="nome" required>



        <label for="email">Email:</label>

        <input type="email" id="email" name="email" required>
```

```
        <label for="mensagem">Mensagem:</label>

        <textarea id="mensagem" name="mensagem" rows="4" cols="50"></textarea>



        <input type="submit" value="Enviar Mensagem">
    </form>

    <h2>Tabela de Horários</h2>
    <table border="1">
        <thead>
            <tr>
                <th>Dia</th>
                <th>Manhã</th>
                <th>Tarde</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Segunda</td>
                <td>Estudar HTML</td>
                <td>Praticar CSS</td>
            </tr>
            <tr>
                <td>Terça</td>
                <td>Estudar CSS</td>
                <td>Praticar JavaScript</td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Explicação do Exemplo

- `<form action="/submit-form" method="post">` : Define um formulário. `action` indica o URL para onde os dados serão enviados (neste caso, um placeholder), e `method="post"` é usado para enviar dados de forma segura.
- `<label for="id_do_input">` : Associa um rótulo a um input, melhorando a acessibilidade.
- `<input type="text" id="nome" name="nome" required>` :
  - `type="text"` : Um campo de texto simples.
  - `id` : Identificador único para o elemento, usado com `label` e JavaScript.

    - `name` : Nome do campo, usado para identificar os dados quando enviados ao servidor.
    - `required` : Torna o campo obrigatório.
- `<input type="email">` : Campo específico para endereços de email, com validação básica.
- `<textarea rows="4" cols="50">` : Uma área de texto para mensagens mais longas.
- `<input type="submit" value="Enviar Mensagem">` : Um botão para submeter o formulário.
- `<table border="1">` : Cria uma tabela com uma borda de 1 pixel.
- `<thead>` , `<tbody>` : Semântica para cabeçalho e corpo da tabela.
- `<tr>` : Define uma linha da tabela.
- `<th>` : Define uma célula de cabeçalho da tabela.
- `<td>` : Define uma célula de dados da tabela.

Exercício do Dia 3

Crie um novo arquivo HTML chamado `cadastro.html` . Neste arquivo, crie um formulário de registo que inclua:

1. Campos para Nome Completo, Email e Senha (com `type="password"`).
2. Um campo de seleção (`<select>`) para escolher o seu nível de experiência em programação (Iniciante, Intermédio, Avançado).
3. Um `checkbox` para aceitar os termos e condições.
4. Um botão de submissão.

Adicione também uma tabela simples que liste 3 cursos online que gostaria de fazer, com colunas para "Nome do Curso" e "Plataforma".

Dia 4: Introdução ao CSS (Seletores e Cores)

Conceitos

O **CSS** (Cascading Style Sheets) é usado para estilizar elementos HTML. Ele descreve como os elementos HTML devem ser exibidos em diferentes meios. Com CSS, pode controlar a cor, fonte, tamanho do texto, espaçamento entre elementos, como os elementos são posicionados e muito mais.

Existem três formas principais de adicionar CSS a um documento HTML:

1. **Inline:** Usando o atributo `style` diretamente na tag HTML (não recomendado para grandes projetos).

2. **Interno:** Dentro da tag `<style>` na secção `<head>` do HTML.
3. **Externo:** Num arquivo `.css` separado, linkado ao HTML através da tag `<link>` na secção `<head>` (a forma mais comum e recomendada).

Os **seletores CSS** são usados para "encontrar" (selecionar) os elementos HTML que deseja estilizar. Os seletores mais comuns são:

- **Seletor de Elemento:** Seleciona todos os elementos de um determinado tipo (ex: `p { ... }` seleciona todos os parágrafos).
- **Seletor de ID:** Seleciona um elemento com um `id` específico (ex: `#meuId { ... }`). IDs devem ser únicos por página.
- **Seletor de Classe:** Seleciona elementos com uma `class` específica (ex: `.minhaClasse { ... }`). Classes podem ser usadas em múltiplos elementos.

Exemplo de Código: `style.css` e `index.html`

Crie um arquivo `style.css` na mesma pasta do seu `index.html`:

CSS

```
/* style.css */

/* Seletor de Elemento */
body {
    font-family: 'Arial', sans-serif;
    background-color: #f4f4f9; /* Cor de fundo cinza claro */
    color: #333; /* Cor do texto principal */
    margin: 20px;
}

/* Seletor de ID */
h1#tituloPrincipal {
    color: #0056b3; /* Azul escuro */
    text-align: center;
}

/* Seletor de Classe */
.paragrafoDestaque {
    color: #e67e22; /* Laranja */
    font-weight: bold;
}

/* Estilo para links */
a {
    color: #27ae60; /* Verde */
    text-decoration: none;
```

```
}

a:hover {
    text-decoration: underline;
}
```

E atualize o seu `index.html` para linkar o CSS e adicionar IDs/Classes:

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Minha Primeira Página Web</title>
    <link rel="stylesheet" href="style.css"> <!-- Link para o arquivo CSS exter
</head>
<body>
    <h1 id="tituloPrincipal">Bem-vindo ao Meu Site!</h1>
    <p class="paragrafoDestaque">Esta é a minha primeira página web. Estou a ap
    <p>Aqui está uma lista de coisas que gosto:</p>
    <ul>
        <li>Programação</li>
        <li>Café</li>
        <li>Música</li>
    </ul>
    <p>E aqui está uma lista numerada:</p>
    <ol>
        <li>Primeiro item</li>
        <li>Segundo item</li>
        <li>Terceiro item</li>
    </ol>

    <h2>Links Úteis</h2>
    <ul>
        <li><a href="https://www.w3schools.com/html/" target="_blank">W3Schools
        <li><a href="https://developer.mozilla.org/pt-BR/docs/Web/HTML" target=
        <li><a href="meu_perfil.html">Ver Meu Perfil (página local )</a></li>
    </ul>

    <h2>Minha Imagem Favorita</h2>
    <img src="https://via.placeholder.com/300x200" alt="Exemplo de imagem place
    <p>Esta é uma imagem de exemplo. O atributo `alt` é importante para acessib

    <h2>Formulário de Contacto</h2>
    <form action="/submit-form" method="post">
        <label for="nome">Nome:</label>
```

```
        <input type="text" id="nome" name="nome" required>



        <label for="email">Email:</label>

        <input type="email" id="email" name="email" required>



        <label for="mensagem">Mensagem:</label>

        <textarea id="mensagem" name="mensagem" rows="4" cols="50"></textarea>



        <input type="submit" value="Enviar Mensagem">
    </form>

    <h2>Tabela de Horários</h2>
    <table border="1">
        <thead>
            <tr>
                <th>Dia</th>
                <th>Manhã</th>
                <th>Tarde</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Segunda</td>
                <td>Estudar HTML</td>
                <td>Praticar CSS</td>
            </tr>
            <tr>
                <td>Terça</td>
                <td>Estudar CSS</td>
                <td>Praticar JavaScript</td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Explicação do Exemplo

- `<link rel="stylesheet" href="style.css">` : Esta linha no `<head>` do HTML conecta o seu documento HTML ao arquivo `style.css`.
- `h1#tituloPrincipal` : Este seletor combina o tipo de elemento (`h1`) com o seu ID (`#tituloPrincipal`).
- `.paragrafoDestaque` : Este seletor aplica estilos a qualquer elemento que tenha a classe `paragrafoDestaque`.
- `a:hover` : É um pseudo-seletor que aplica estilos quando o cursor do rato passa por cima de um link.
- **Cores:** Pode usar nomes de cores (ex: `red`), valores hexadecimais (ex: `#FF0000`), ou funções RGB/RGBA (ex: `rgb(255, 0, 0)`).

Exercício do Dia 4

1. Crie um novo arquivo CSS chamado `meu_estilo.css` na mesma pasta do seu `meu_perfil.html` e linke-o ao `meu_perfil.html`.
2. No `meu_estilo.css`, adicione estilos para:
   - Mudar a cor de fundo do `body` para um tom suave de azul ou verde.
   - Centralizar o seu nome (`<h1>`) e mudar a sua cor.
   - Mudar a cor do texto dos seus hobbies (`<li>` dentro de `<ul>`) para uma cor diferente.
   - Estilizar os links para que não tenham sublinhado e mudem de cor ao passar o rato.

Dia 5: O Box Model (Margens, Bordas, Padding)

Conceitos

No CSS, cada elemento HTML é tratado como uma caixa retangular. O **Box Model** descreve como estas caixas são estruturadas e como o espaço é ocupado. Ele consiste em quatro partes, de dentro para fora:

1. **Content (Conteúdo):** Onde o conteúdo real (texto, imagens) do elemento é exibido.
2. **Padding (Preenchimento):** O espaço entre o conteúdo e a borda do elemento. É o "estofamento" interno da caixa.
3. **Border (Borda):** A linha que envolve o padding e o conteúdo.
4. **Margin (Margem):** O espaço transparente fora da borda, que separa o elemento de outros elementos vizinhos.

Compreender o Box Model é fundamental para controlar o layout e o espaçamento dos elementos na sua página.

Exemplo de Código: `style.css`

```CSS
/* style.css */

body {
    font-family: 'Arial', sans-serif;
    background-color: #f4f4f9;
    color: #333;
    margin: 0; /* Remover margem padrão do body */
    padding: 20px; /* Adicionar um padding geral à página */
}

h1 {
    background-color: #e0e0e0;
    padding: 15px; /* Espaço entre o texto e a borda do h1 */
    border: 2px solid #555; /* Borda sólida de 2px cinza escuro */
    margin-bottom: 30px; /* Espaço abaixo do h1 */
    text-align: center;
}

p {
    background-color: #ffffff;
    padding: 10px 15px; /* Padding superior/inferior de 10px, esquerdo/direito
    border: 1px dashed #ccc; /* Borda tracejada cinza claro */
    margin: 10px 0; /* Margem superior/inferior de 10px, esquerda/direita de 0
}

.caixa-especial {
    width: 200px;
    height: 100px;
    background-color: lightblue;
    padding: 20px;
    border: 5px solid blue;
    margin: 30px auto; /* Margem superior/inferior de 30px, 'auto' centraliza h
}
```

Explicação do Exemplo

- `margin: 0; padding: 20px;` no `body`: É comum "resetar" a margem padrão do `body` e adicionar um padding geral para afastar o conteúdo das bordas da janela.
- `padding: 15px;` no `h1`: Adiciona 15 pixels de espaço entre o texto "Bem-vindo ao Meu Site!" e a borda do `h1`.

- `border: 2px solid #555;` : Define uma borda de 2 pixels de espessura, estilo sólido e cor cinza escuro.
- `margin-bottom: 30px;` : Cria um espaço de 30 pixels abaixo do `h1`.
- `padding: 10px 15px;` : Quando dois valores são dados, o primeiro é para cima/baixo e o segundo para esquerda/direita.
- `margin: 10px 0;` : Similar ao padding, o primeiro valor é para cima/baixo e o segundo para esquerda/direita.
- `margin: 30px auto;` : `auto` para as margens esquerda/direita, combinado com uma `width` definida, centraliza o elemento horizontalmente na página.

Exercício do Dia 5

1. No seu `style.css` (ou `meu_estilo.css`), selecione os elementos `<ul>` e `<li>`.
2. Adicione um `padding` de 10px aos `<li>`.
3. Adicione uma `border` de 1px sólida e cinza claro aos `<li>`.
4. Adicione uma `margin-bottom` de 5px a cada `<li>` para separá-los um pouco.
5. Crie uma nova `div` no seu `index.html` (ou `meu_perfil.html`) com a classe `cartao-produto`.
6. No seu CSS, estilize `cartao-produto` com:
   - `width: 300px;`
   - `padding: 20px;`
   - `border: 1px solid #ddd;`
   - `margin: 20px auto;` (para centralizar)
   - `background-color: #fff;`
   - `box-shadow: 2px 2px 5px rgba(0,0,0,0.1);` (para dar um efeito de profundidade).

Dia 6: Posicionamento e Layout Simples

Conceitos

O posicionamento de elementos é crucial para criar layouts complexos. O CSS oferece a propriedade `position` com vários valores:

- **`static` (Padrão):** Os elementos são posicionados de acordo com o fluxo normal do documento.
- **`relative`:** O elemento é posicionado de acordo com o fluxo normal, mas pode ser deslocado da sua posição original usando `top`, `right`, `bottom`, `left`. O espaço original

que ocuparia permanece reservado.

- **`absolute`:** O elemento é removido do fluxo normal do documento e posicionado em relação ao seu ancestral posicionado mais próximo (não `static`). Se não houver ancestral posicionado, ele é posicionado em relação ao `<body>`.
- **`fixed`:** O elemento é removido do fluxo normal e posicionado em relação à janela de visualização (viewport). Ele permanece na mesma posição mesmo quando a página é rolada (útil para cabeçalhos ou rodapés fixos).
- **`sticky`:** O elemento é posicionado como `relative` até que a sua posição de rolagem atinja um limite especificado, quando então se torna `fixed` (útil para menus que "grudam" no topo ao rolar).

Além de `position`, propriedades como `float` e `display` (especialmente `block`, `inline`, `inline-block`) são usadas para controlar o fluxo e o layout dos elementos.

- **`display: block;`:** Ocupa toda a largura disponível, começa numa nova linha (ex: `div`, `p`, `h1`).
- **`display: inline;`:** Ocupa apenas a largura necessária, não começa numa nova linha (ex: `span`, `a`, `img`).
- **`display: inline-block;`:** Combina características de `inline` (não quebra linha) e `block` (pode ter `width`, `height`, `margin`, `padding`).
- **`float: left;` / `float: right;`:** Tira o elemento do fluxo normal e o empurra para a esquerda ou direita, permitindo que outros elementos "flutuem" ao seu redor. Requer `clear: both;` para limpar o float em elementos subsequentes.

Exemplo de Código: `style.css` e `index.html`

Atualize o seu `style.css`:

CSS

```
/* style.css */

/* ... estilos anteriores ... */

.header-fixo {
    position: fixed; /* Fixo na viewport */
    top: 0;
    left: 0;
    width: 100%;
    background-color: #333;
    color: white;
    padding: 10px 0;
    text-align: center;
```

```
    z-index: 1000; /* Garante que fique por cima de outros elementos */
}

.conteudo-principal {
    margin-top: 70px; /* Para evitar que o conteúdo fique por baixo do header f
    padding: 20px;
}

.caixa-flutuante {
    float: left;
    width: 150px;
    height: 100px;
    background-color: #f0f0f0;
    border: 1px solid #ccc;
    margin: 10px;
    padding: 10px;
}

.clear-float {
    clear: both; /* Limpa os floats anteriores */
}

.elemento-relativo {
    position: relative;
    top: 20px;
    left: 20px;
    background-color: lightcoral;
    width: 100px;
    height: 50px;
}

.elemento-absoluto {
    position: absolute;
    top: 10px;
    right: 10px;
    background-color: lightgreen;
    width: 80px;
    height: 40px;
}
```

E atualize o seu `index.html`:

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Minha Primeira Página Web</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header class="header-fixo">
        <h1>Meu Site Fixo</h1>
    </header>

    <div class="conteudo-principal">
        <h1 id="tituloPrincipal">Bem-vindo ao Meu Site!</h1>
        <p class="paragrafoDestaque">Esta é a minha primeira página web. Estou 

        <div class="caixa-flutuante">Caixa 1</div>
        <div class="caixa-flutuante">Caixa 2</div>
        <div class="caixa-flutuante">Caixa 3</div>
        <div class="clear-float"></div> <!-- Elemento para limpar o float -->

        <p>Este parágrafo vem depois das caixas flutuantes.</p>

        <div class="elemento-relativo">
            Relativo
            <div class="elemento-absoluto">Absoluto</div>
        </div>

        <p>Mais conteúdo aqui para testar o scroll e o header fixo.</p>
        <!-- Adicione mais parágrafos para forçar o scroll -->
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eius
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eius
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eius
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eius
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eius
    </div>
</body>
</html>
```

Explicação do Exemplo

- `.header-fixo` : Usa `position: fixed;` para manter o cabeçalho visível no topo da página, independentemente da rolagem.
- `.conteudo-principal` : Tem uma `margin-top` para garantir que o conteúdo não fique escondido atrás do cabeçalho fixo.
- `.caixa-flutuante` : Demonstra o uso de `float: left;` para dispor elementos lado a lado.
- `.clear-float` : Uma `div` vazia com `clear: both;` é uma técnica comum para garantir que os elementos seguintes não sejam afetados pelos floats.

- `.elemento-relativo` e `.elemento-absoluto`: Mostra como um elemento com `position: absolute;` é posicionado em relação ao seu ancestral `relative`.

Exercício do Dia 6

1. Crie um layout de duas colunas usando `float` no seu `index.html` (ou num novo arquivo `layout.html`). Uma coluna deve ter 70% da largura e a outra 30%.
2. Adicione um rodapé fixo na parte inferior da página, com um texto simples como "Direitos Reservados © 2026".
3. Dentro da coluna de 70%, crie uma `div` com `position: relative;` e, dentro dela, uma pequena `div` com `position: absolute;` que fique no canto superior direito.

Dia 7: Projeto 1: Currículo Online

Conceitos

Este dia é dedicado a consolidar todo o conhecimento adquirido na primeira semana através da construção de um projeto prático. O objetivo é criar um currículo online completo, utilizando apenas HTML para a estrutura e CSS para o estilo e layout.

O projeto deve incluir:

- **Informações Pessoais:** Nome, contacto, título profissional.
- **Resumo/Sobre Mim:** Um parágrafo a descrever as suas qualificações e objetivos.
- **Experiência Profissional:** Uma lista de empregos anteriores com datas, cargos e responsabilidades (pode usar listas ou tabelas).
- **Educação:** Detalhes sobre a sua formação académica.
- **Habilidades:** Uma lista de competências (ex: HTML, CSS, Comunicação).
- **Links:** Para o seu LinkedIn, GitHub, portfólio (se tiver).

Requisitos Técnicos

- Utilize tags HTML semânticas (`<header>`, `<main>`, `<footer>`, `<section>`, `<article>`, etc.) sempre que apropriado.
- Aplique estilos CSS para tornar o currículo visualmente apelativo e fácil de ler.
- Use o Box Model para controlar o espaçamento e as margens.
- Experimente com diferentes propriedades de posicionamento e display para organizar as secções.
- Não se preocupe com a responsividade ainda, focaremos nisso na próxima semana.

Exercício do Dia 7

Crie um novo arquivo `curriculo.html` e um `curriculo.css`.

1. Estruture o seu currículo online usando as tags HTML aprendidas.
2. Estilize-o com CSS, definindo cores, fontes, tamanhos, margens, paddings e bordas.
3. Organize as secções de forma clara e profissional. Pode usar `float` para criar um layout de duas colunas para algumas secções (ex: informações de contacto ao lado do resumo).
4. Certifique-se de que todos os links funcionam (mesmo que apontem para placeholders).

Dica: Procure inspiração em modelos de currículos online, mas tente implementar o seu próprio design e estrutura.

Semana 2: Layouts Modernos e Responsividade (O Site que se Adapta)

Dia 8: Flexbox: O Guia Definitivo

Conceitos

O **Flexbox** (Flexible Box Layout Module) é um modelo de layout unidimensional que permite organizar itens num contentor em linhas ou colunas. É extremamente poderoso para distribuir espaço e alinhar itens de forma eficiente, tornando o design responsivo muito mais fácil.

Quando se define `display: flex;` num elemento, ele torna-se um **contentor flex** (flex container). Os seus filhos diretos tornam-se **itens flex** (flex items).

Propriedades do **Contentor Flex**:

- `flex-direction`: Define a direção principal em que os itens são colocados (ex: `row`, `column`).
- `justify-content`: Alinha os itens ao longo do eixo principal (ex: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`).
- `align-items`: Alinha os itens ao longo do eixo transversal (ex: `flex-start`, `flex-end`, `center`, `stretch`).
- `flex-wrap`: Controla se os itens flex devem quebrar para a próxima linha (ex: `nowrap`, `wrap`).

- `align-content`: Alinha as linhas flex quando há várias linhas (ex: `flex-start`, `flex-end`, `center`).

Propriedades dos **Itens Flex**:

- `flex-grow`: Define a capacidade de um item crescer se houver espaço extra.
- `flex-shrink`: Define a capacidade de um item encolher se não houver espaço suficiente.
- `flex-basis`: Define o tamanho inicial de um item antes que o espaço restante seja distribuído.
- `flex`: É uma abreviação para `flex-grow`, `flex-shrink` e `flex-basis`.
- `order`: Define a ordem em que um item aparece no contentor flex.
- `align-self`: Permite alinhar um item flex individualmente ao longo do eixo transversal.

Exemplo de Código: `flexbox.html` e `style.css`

Crie um novo arquivo `flexbox.html`:

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Exemplo de Flexbox</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Exemplo de Flexbox</h1>

    <h2>Barra de Navegação com Flexbox</h2>
    <nav class="navbar">
        <a href="#">Home</a>
        <a href="#">Sobre</a>
        <a href="#">Serviços</a>
        <a href="#">Contacto</a>
    </nav>

    <h2>Galeria de Imagens Flexível</h2>
    <div class="galeria-flex">
        <div class="item-galeria">Imagem 1</div>
        <div class="item-galeria">Imagem 2</div>
        <div class="item-galeria">Imagem 3</div>
        <div class="item-galeria">Imagem 4</div>
        <div class="item-galeria">Imagem 5</div>
```

```
    </div>

    <h2>Layout de Colunas</h2>
    <div class="layout-colunas">
        <div class="coluna">Coluna 1</div>
        <div class="coluna">Coluna 2</div>
        <div class="coluna">Coluna 3</div>
    </div>
</body>
</html>
```

E adicione ao seu `style.css`:

CSS

```
/* style.css */

/* ... estilos anteriores ... */

.navbar {
    display: flex; /* Torna o contentor flex */
    justify-content: space-around; /* Distribui o espaço igualmente entre os it
    align-items: center; /* Centraliza os itens verticalmente */
    background-color: #444;
    padding: 10px;
    margin-bottom: 20px;
}

.navbar a {
    color: white;
    text-decoration: none;
    padding: 5px 10px;
}

.galeria-flex {
    display: flex;
    flex-wrap: wrap; /* Permite que os itens quebrem para a próxima linha */
    justify-content: center;
    gap: 10px; /* Espaçamento entre os itens */
    margin-bottom: 20px;
}

.item-galeria {
    background-color: #eee;
    border: 1px solid #ccc;
    padding: 20px;
    width: 150px; /* Largura base */
    text-align: center;
```

```
    flex-grow: 1; /* Permite que os itens cresçam para preencher o espaço */
    max-width: 200px; /* Limite máximo de largura */
}

.layout-colunas {
    display: flex;
    gap: 15px;
}

.coluna {
    background-color: #d4edda;
    border: 1px solid #28a745;
    padding: 20px;
    flex: 1; /* Abreviação para flex-grow: 1, flex-shrink: 1, flex-basis: 0% */
}
```

Explicação do Exemplo

- `.navbar`: Usa `display: flex;` para criar uma barra de navegação horizontal. `justify-content: space-around;` distribui os links uniformemente, e `align-items: center;` os centraliza verticalmente.
- `.galeria-flex`: `flex-wrap: wrap;` é crucial para que os itens da galeria se ajustem a diferentes tamanhos de ecrã, quebrando para a linha seguinte quando necessário. `gap` adiciona espaçamento entre os itens.
- `.item-galeria`: `flex-grow: 1;` permite que os itens se expandam para preencher o espaço disponível, mas `max-width` impede que fiquem demasiado grandes.
- `.layout-colunas`: `flex: 1;` em cada coluna faz com que todas as colunas tenham a mesma largura e se ajustem ao espaço disponível.

Exercício do Dia 8

1. Crie um novo arquivo `dashboard.html` e `dashboard.css`.
2. No `dashboard.html`, crie um cabeçalho com um título e um menu de navegação à direita, usando Flexbox para alinhar os itens.
3. Abaixo do cabeçalho, crie uma secção principal com três "cards" de informação dispostos horizontalmente. Cada card deve ter um título e um parágrafo. Use Flexbox para que os cards se distribuam uniformemente e quebrem para a linha seguinte em ecrãs menores.
4. Adicione um rodapé simples, também alinhado com Flexbox.

Dia 9: CSS Grid Layout

Conceitos

O **CSS Grid Layout** é um sistema de layout bidimensional para a web. Ao contrário do Flexbox (que é unidimensional, linha OU coluna), o Grid permite organizar elementos em linhas E colunas simultaneamente, tornando-o ideal para layouts de página inteira e estruturas complexas.

Quando se define `display: grid;` num elemento, ele torna-se um **contentor grid** (grid container). Os seus filhos diretos tornam-se **itens grid** (grid items).

Propriedades do **Contentor Grid**:

- `grid-template-columns` : Define o número e a largura das colunas (ex: `grid-template-columns: 1fr 1fr 1fr;` para 3 colunas de largura igual).
- `grid-template-rows` : Define o número e a altura das linhas.
- `grid-gap` (ou `gap`): Define o espaçamento entre as linhas e colunas (ex: `gap: 10px;`).
- `justify-items` : Alinha os itens dentro das células ao longo do eixo da linha.
- `align-items` : Alinha os itens dentro das células ao longo do eixo da coluna.
- `grid-template-areas` : Permite nomear áreas do grid e depois posicionar os itens usando esses nomes, o que torna o layout muito legível.

Propriedades dos **Itens Grid**:

- `grid-column-start` , `grid-column-end` , `grid-row-start` , `grid-row-end` : Definem onde um item começa e termina no grid.
- `grid-column` e `grid-row` : Abreviações para as propriedades acima.
- `grid-area` : Usado em conjunto com `grid-template-areas` para posicionar um item numa área nomeada.

Exemplo de Código: `grid.html` e `style.css`

Crie um novo arquivo `grid.html` :

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Exemplo de CSS Grid</title>
    <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
    <h1>Exemplo de CSS Grid Layout</h1>

    <h2>Layout de Página com Grid Areas</h2>
    <div class="layout-pagina">
        <header class="header-grid">Cabeçalho</header>
        <nav class="nav-grid">Navegação</nav>
        <main class="main-grid">Conteúdo Principal</main>
        <aside class="sidebar-grid">Sidebar</aside>
        <footer class="footer-grid">Rodapé</footer>
    </div>

    <h2>Galeria de Imagens com Grid</h2>
    <div class="galeria-grid">
        <div class="item-grid">1</div>
        <div class="item-grid">2</div>
        <div class="item-grid">3</div>
        <div class="item-grid">4</div>
        <div class="item-grid">5</div>
        <div class="item-grid">6</div>
    </div>
</body>
</html>
```

E adicione ao seu `style.css`:

CSS

```
/* style.css */

/* ... estilos anteriores ... */

.layout-pagina {
    display: grid;
    grid-template-columns: 1fr 3fr 1fr; /* Coluna esquerda, conteúdo, coluna di
    grid-template-rows: auto 1fr auto; /* Cabeçalho, conteúdo principal, rodapé
    grid-template-areas:
        "header header header"
        "nav    main   sidebar"
        "footer footer footer";
    gap: 10px;
    min-height: 80vh;
    border: 2px solid #007bff;
    padding: 10px;
}

.header-grid {
```

```
    grid-area: header;
    background-color: #007bff;
    color: white;
    padding: 15px;
    text-align: center;
}

.nav-grid {
    grid-area: nav;
    background-color: #6c757d;
    color: white;
    padding: 15px;
}

.main-grid {
    grid-area: main;
    background-color: #e9ecef;
    padding: 15px;
}

.sidebar-grid {
    grid-area: sidebar;
    background-color: #6c757d;
    color: white;
    padding: 15px;
}

.footer-grid {
    grid-area: footer;
    background-color: #343a40;
    color: white;
    padding: 15px;
    text-align: center;
}

.galeria-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); /* Colunas res
    gap: 10px;
    margin-top: 30px;
    border: 2px solid #28a745;
    padding: 10px;
}

.item-grid {
    background-color: #d4edda;
    border: 1px solid #28a745;
    padding: 20px;
```

```
    text-align: center;
}
```

Explicação do Exemplo

- `.layout-pagina`: Define um grid com 3 colunas e 3 linhas. `1fr` significa uma fração do espaço disponível. `auto` ajusta-se ao conteúdo.
- `grid-template-areas`: Permite criar um layout visualmente, nomeando as áreas. Isso torna o código muito mais fácil de entender.
- `grid-area: header;`: Atribui o elemento com a classe `header-grid` à área nomeada `header`.
- `.galeria-grid`: Usa `repeat(auto-fit, minmax(150px, 1fr))` para criar uma galeria responsiva. As colunas terão no mínimo 150px e no máximo 1 fração do espaço disponível, ajustando-se automaticamente ao número de colunas.

Exercício do Dia 9

1. Crie um novo arquivo `revista.html` e `revista.css`.
2. Desenhe um layout de revista usando CSS Grid. O layout deve ter:
   - Um cabeçalho que ocupe toda a largura.
   - Uma secção principal com duas colunas para artigos.
   - Uma barra lateral à direita para anúncios ou links relacionados.
   - Um rodapé que ocupe toda a largura.
3. Use `grid-template-areas` para definir o layout de forma clara.
4. Preencha as áreas com conteúdo placeholder (títulos, parágrafos, imagens).

Dia 10: Media Queries e Design Responsivo

Conceitos

O **Design Responsivo** é uma abordagem de design web que faz com que os websites se adaptem a diferentes tamanhos de ecrã e dispositivos (desktops, tablets, telemóveis). O objetivo é proporcionar uma experiência de utilizador ótima, independentemente do dispositivo.

As **Media Queries** são a ferramenta fundamental do CSS para implementar o design responsivo. Elas permitem aplicar estilos CSS diferentes com base nas características do dispositivo, como largura do ecrã, altura, orientação, etc.

A sintaxe básica de uma Media Query é:

```
CSS

@media screen and (max-width: 768px) {
    /* Estilos aplicados quando a largura do ecrã é no máximo 768px */
}

@media screen and (min-width: 769px) and (max-width: 1024px) {
    /* Estilos aplicados para ecrãs entre 769px e 1024px */
}
```

- `@media` : Palavra-chave para iniciar uma media query.
- `screen` : Tipo de média (para ecrãs de computador, tablets, smartphones).
- `and` : Operador lógico para combinar múltiplas condições.
- `max-width` : Aplica estilos quando a largura do ecrã é menor ou igual ao valor especificado (abordagem "mobile-first" ou "desktop-first").
- `min-width` : Aplica estilos quando a largura do ecrã é maior ou igual ao valor especificado.

É uma boa prática começar com o design para telemóveis (mobile-first) e depois adicionar media queries para ecrãs maiores. Isso garante que a experiência básica é funcional em dispositivos móveis e melhora progressivamente para ecrãs maiores.

Exemplo de Código: `responsive.html` e `style.css`

Crie um novo arquivo `responsive.html` :

```
HTML

<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Design Responsivo</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header class="cabecalho-responsivo">
        <h1>Meu Site Responsivo</h1>
        <nav class="menu-responsivo">
            <a href="#">Link 1</a>
            <a href="#">Link 2</a>
            <a href="#">Link 3</a>
        </nav>
    </header>
```

```
    <div class="conteudo-responsivo">
        <div class="coluna-responsiva">
            <h2>Coluna 1</h2>
            <p>Este é o conteúdo da primeira coluna. Em ecrãs pequenos, as colu
        </div>
        <div class="coluna-responsiva">
            <h2>Coluna 2</h2>
            <p>Este é o conteúdo da segunda coluna. Em ecrãs pequenos, as colun
        </div>
        <div class="coluna-responsiva">
            <h2>Coluna 3</h2>
            <p>Este é o conteúdo da terceira coluna. Em ecrãs pequenos, as colu
        </div>
    </div>

    <footer class="rodape-responsivo">
        <p>Rodapé Responsivo &copy; 2026</p>
    </footer>
</body>
</html>
```

E adicione ao seu `style.css`:

CSS

```
/* style.css */

/* ... estilos anteriores ... */

/* Estilos base (Mobile-First) */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f8f9fa;
    color: #343a40;
}

.cabecalho-responsivo {
    background-color: #007bff;
    color: white;
    padding: 15px;
    text-align: center;
}

.menu-responsivo a {
    display: block; /* Links empilhados em mobile */
```

```
    color: white;
    text-decoration: none;
    padding: 10px;
    border-bottom: 1px solid #0056b3;
}

.menu-responsivo a:last-child {
    border-bottom: none;
}

.conteudo-responsivo {
    padding: 20px;
}

.coluna-responsiva {
    background-color: #ffffff;
    border: 1px solid #dee2e6;
    margin-bottom: 20px;
    padding: 15px;
}

.rodape-responsivo {
    background-color: #343a40;
    color: white;
    text-align: center;
    padding: 15px;
}

/* Media Query para Tablets (largura mínima de 768px) */
@media screen and (min-width: 768px) {
    .menu-responsivo a {
        display: inline-block; /* Links lado a lado em tablets */
        border-bottom: none;
        margin: 0 10px;
    }

    .conteudo-responsivo {
        display: flex; /* Colunas lado a lado em tablets */
        gap: 20px;
    }

    .coluna-responsiva {
        flex: 1; /* Distribui o espaço igualmente */
        margin-bottom: 0;
    }
}

/* Media Query para Desktops (largura mínima de 1024px) */
```

```
@media screen and (min-width: 1024px) {
    .cabecalho-responsivo {
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .menu-responsivo {
        margin-right: 20px;
    }

    .conteudo-responsivo {
        max-width: 1200px;
        margin: 0 auto;
    }
}
```

Explicação do Exemplo

- **Mobile-First:** Os estilos básicos são definidos para ecrãs pequenos (telemóveis). Por exemplo, os links do menu são `display: block;` para empilharem-se.
- `@media screen and (min-width: 768px)` : Quando a largura do ecrã atinge 768px (geralmente tablets), os links do menu tornam-se `inline-block` e as colunas do conteúdo usam `display: flex;` para ficarem lado a lado.
- `@media screen and (min-width: 1024px)` : Para ecrãs maiores (desktops), o cabeçalho usa Flexbox para alinhar o título e o menu em lados opostos, e o conteúdo principal tem uma `max-width` e é centralizado.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` : Esta meta tag no HTML é **essencial** para o design responsivo, pois informa ao navegador para definir a largura da viewport para a largura do dispositivo e a escala inicial para 1.0.

Exercício do Dia 10

1. Pegue no seu projeto de currículo online do Dia 7 (`curriculo.html` e `curriculo.css`).
2. Torne-o responsivo usando Media Queries.
    - Em ecrãs pequenos (até 767px), as secções devem empilhar-se verticalmente, e o texto deve ser facilmente legível.
    - Em tablets (768px a 1023px), pode manter um layout de uma coluna ou tentar um layout de duas colunas mais simples.
    - Em desktops (a partir de 1024px), o layout original de duas colunas (se tiver usado `float`) deve ser mantido ou melhorado com Flexbox/Grid.

3. Teste o seu currículo redimensionando a janela do navegador ou usando as ferramentas de programador (modo de dispositivo).

Dia 11: Variáveis CSS e Arquitetura

Conceitos

As **Variáveis CSS** (também conhecidas como Propriedades Personalizadas CSS ou CSS Custom Properties) permitem definir valores reutilizáveis no seu CSS. Isso torna o código mais fácil de manter, mais legível e mais flexível. São definidas com dois hífens `--` e acedidas com a função `var()`.

```CSS
:root {
    --cor-primaria: #007bff;
    --cor-secundaria: #6c757d;
    --espacamento-padrao: 15px;
}

h1 {
    color: var(--cor-primaria);
    padding: var(--espacamento-padrao);
}
```

Arquitetura CSS refere-se à forma como organiza o seu código CSS para que seja escalável, fácil de manter e compreensível. Algumas metodologias populares incluem:

- **BEM (Block, Element, Modifier):** Uma convenção de nomenclatura que ajuda a criar classes CSS claras e modulares (ex: `.bloco__elemento--modificador`).
- **SMACSS (Scalable and Modular Architecture for CSS):** Organiza o CSS em categorias (Base, Layout, Modules, State, Theme).
- **ITCSS (Inverted Triangle CSS):** Organiza o CSS por especificidade, do mais genérico ao mais específico.

Para iniciantes, o mais importante é manter o código organizado em arquivos separados (seções, componentes) e usar comentários claros.

Exemplo de Código: `style.css`

```CSS
/* style.css */
```

```
:root {
    --cor-primaria: #007bff;
    --cor-secundaria: #6c757d;
    --cor-texto-claro: #f8f9fa;
    --cor-texto-escuro: #343a40;
    --espacamento-pequeno: 10px;
    --espacamento-medio: 20px;
    --fonte-padrao: 'Arial', sans-serif;
}

body {
    font-family: var(--fonte-padrao);
    background-color: var(--cor-texto-claro);
    color: var(--cor-texto-escuro);
    margin: 0;
    padding: var(--espacamento-medio);
}

.cabecalho {
    background-color: var(--cor-primaria);
    color: var(--cor-texto-claro);
    padding: var(--espacamento-medio);
    text-align: center;
}

.botao-primario {
    background-color: var(--cor-primaria);
    color: white;
    padding: var(--espacamento-pequeno) var(--espacamento-medio);
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

.botao-primario:hover {
    background-color: #0056b3; /* Um tom mais escuro da cor primária */
}

/* Exemplo de BEM */
.card {
    background-color: white;
    border: 1px solid #ddd;
    padding: var(--espacamento-medio);
    margin-bottom: var(--espacamento-medio);
    border-radius: 8px;
}
```

```
.card__titulo {
    color: var(--cor-primaria);
    font-size: 1.5em;
    margin-bottom: var(--espacamento-pequeno);
}

.card__texto {
    line-height: 1.6;
}

.card--destaque {
    border-color: var(--cor-secundaria);
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
```

Explicação do Exemplo

- `:root` : Onde as variáveis globais são definidas. `:root` é o seletor de mais alto nível e abrange todo o documento HTML.
- `--cor-primaria: #007bff;` : Define uma variável chamada `--cor-primaria` com o valor `#007bff`.
- `color: var(--cor-primaria);` : Acede o valor da variável `--cor-primaria`.
- **BEM:** O exemplo do `.card`, `.card__titulo`, `.card__texto` e `.card--destaque` ilustra a convenção BEM, onde `__` separa o bloco do elemento e `--` separa o bloco/elemento do modificador.

Exercício do Dia 11

1. Revise o seu `curriculo.css` (do Dia 7/10).
2. Defina pelo menos 5 variáveis CSS no seletor `:root` para cores, tamanhos de fonte e espaçamentos que usa repetidamente.
3. Substitua os valores diretos no seu CSS pelas variáveis correspondentes.
4. Crie uma nova secção no seu currículo (ex: "Projetos") e use a metodologia BEM para nomear as classes CSS dos elementos dentro dessa secção (ex: `.projeto`, `.projeto__titulo`, `.projeto__descricao`, `.projeto__link`).

Dia 12: Transições e Animações

Conceitos

Transições CSS permitem mudar suavemente o valor de uma propriedade CSS de um estado para outro durante um período de tempo. São ideais para efeitos simples de

interação, como mudar a cor de um botão ao passar o rato.

Propriedades de transição:

- `transition-property`: A propriedade CSS à qual a transição será aplicada (ex: `background-color`, `all`).
- `transition-duration`: A duração da transição (ex: `0.3s`, `500ms`).
- `transition-timing-function`: A curva de velocidade da transição (ex: `ease`, `linear`, `ease-in-out`).
- `transition-delay`: Um atraso antes do início da transição.
- `transition`: Abreviação para todas as propriedades acima.

Animações CSS são mais complexas e permitem criar sequências de estilos que mudam ao longo do tempo. São definidas usando `@keyframes` e depois aplicadas a um elemento.

Propriedades de animação:

- `@keyframes nomeDaAnimacao { from { ... } to { ... } }` ou `{ 0% { ... } 50% { ... } 100% { ... } }`.
- `animation-name`: O nome da animação definida com `@keyframes`.
- `animation-duration`: A duração de um ciclo da animação.
- `animation-timing-function`: A curva de velocidade.
- `animation-delay`: Um atraso antes do início da animação.
- `animation-iteration-count`: Quantas vezes a animação deve ser repetida (ex: `infinite`).
- `animation-direction`: Se a animação deve ser reproduzida para a frente, para trás ou alternadamente.
- `animation-fill-mode`: Define o estilo aplicado ao elemento antes e depois da execução da animação.
- `animation`: Abreviação para todas as propriedades acima.

Exemplo de Código: `animations.html` e `style.css`

Crie um novo arquivo `animations.html`:

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Transições e Animações CSS</title>
    <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
    <h1>Transições e Animações</h1>

    <h2>Exemplo de Transição</h2>
    <div class="caixa-transicao">Passe o rato</div>

    <h2>Exemplo de Animação</h2>
    <div class="caixa-animacao"></div>

    <h2>Botão com Efeito Hover</h2>
    <button class="botao-animado">Clique-me!</button>
</body>
</html>
```

E adicione ao seu `style.css`:

CSS

```
/* style.css */

/* ... estilos anteriores ... */

.caixa-transicao {
    width: 100px;
    height: 100px;
    background-color: #3498db;
    margin: 20px;
    transition: background-color 0.5s ease-in-out, transform 0.5s ease-in-out;
}

.caixa-transicao:hover {
    background-color: #e74c3c;
    transform: rotate(45deg) scale(1.2); /* Rotação e aumento de tamanho */
}

@keyframes pulsar {
    0%   { transform: scale(1); opacity: 1; }
    50%  { transform: scale(1.1); opacity: 0.7; }
    100% { transform: scale(1); opacity: 1; }
}

.caixa-animacao {
    width: 80px;
    height: 80px;
    background-color: #2ecc71;
    margin: 20px;
    border-radius: 50%;
```

```
    animation-name: pulsar;
    animation-duration: 2s;
    animation-iteration-count: infinite;
    animation-timing-function: ease-in-out;
}

.botao-animado {
    background-color: #f39c12;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    margin: 20px;
    transition: all 0.3s ease;
}

.botao-animado:hover {
    background-color: #e67e22;
    transform: translateY(-3px); /* Move ligeiramente para cima */
    box-shadow: 0 5px 10px rgba(0,0,0,0.2);
}
```

Explicação do Exemplo

- `.caixa-transicao`: A transição é aplicada quando o estado do `:hover` é ativado. Note que pode transicionar múltiplas propriedades separando-as por vírgulas.
- `@keyframes pulsar`: Define uma animação chamada `pulsar` que muda a escala e a opacidade do elemento em diferentes pontos da animação.
- `.caixa-animacao`: Aplica a animação `pulsar` com uma duração de 2 segundos e repetição infinita.
- `.botao-animado`: Demonstra uma transição mais complexa no `hover`, incluindo `transform` e `box-shadow`.

Exercício do Dia 12

1. No seu projeto de currículo (`curriculo.html` e `curriculo.css`), adicione transições a alguns elementos:
    - Faça com que os links mudem de cor e/ou fiquem ligeiramente maiores ao passar o rato.
    - Se tiver botões, adicione um efeito de `hover` com transição para a cor de fundo e um pequeno `transform`.

2. Crie uma pequena animação de "carregamento" (ex: um círculo que gira ou pulsa) e adicione-a a uma secção do seu currículo (pode ser uma secção "A Carregar Habilidades" que desaparece depois).

Dia 13: Acessibilidade e SEO Básico

Conceitos

Acessibilidade Web (A11y) significa garantir que os websites e aplicações web podem ser usados por pessoas com a mais ampla gama de habilidades e deficiências. Isso inclui pessoas com deficiências visuais, auditivas, motoras e cognitivas. É um aspeto crucial do desenvolvimento web e, em muitos países, é um requisito legal.

Princípios chave da acessibilidade:

- **Percebível:** A informação e os componentes da interface do utilizador devem ser apresentados aos utilizadores de formas que eles possam perceber.
- **Operável:** Os componentes da interface do utilizador e a navegação devem ser operáveis.
- **Compreensível:** A informação e a operação da interface do utilizador devem ser compreensíveis.
- **Robusto:** O conteúdo deve ser robusto o suficiente para ser interpretado por uma ampla variedade de agentes de utilizador, incluindo tecnologias de assistência.

SEO (Search Engine Optimization) é o processo de otimizar o seu website para obter um ranking mais alto nos resultados dos motores de busca (como Google, Bing). Um bom SEO aumenta a visibilidade do seu site e atrai mais tráfego orgânico.

Práticas básicas de SEO:

- **Tags de Título (`<title>`):** Devem ser únicas, descritivas e conter palavras-chave relevantes.
- **Meta Descrição (`<meta name="description">`):** Um resumo conciso do conteúdo da página, que aparece nos resultados de pesquisa.
- **Tags de Cabeçalho (`<h1>` a `<h6>`):** Use-as para estruturar o conteúdo de forma lógica, com `<h1>` para o título principal da página.
- **Atributos `alt` em Imagens:** Descrevem o conteúdo da imagem para motores de busca e leitores de ecrã.
- **URLs Amigáveis:** URLs curtos, descritivos e que contenham palavras-chave.
- **Conteúdo de Qualidade:** Conteúdo relevante, original e útil para o utilizador.

- **Links Internos e Externos:** Links para outras páginas do seu site e para sites externos de autoridade.
- **Responsividade:** Sites responsivos são preferidos pelos motores de busca.

Exemplo de Código: `seo_a11y.html`

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dicas de Acessibilidade e SEO - Meu Site</title>
    <meta name="description" content="Aprenda sobre acessibilidade web e otimiz
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <h1>Melhorando a Acessibilidade e SEO do Seu Site</h1>
        <nav aria-label="Navegação Principal">
            <ul>
                <li><a href="#conteudo-principal">Ir para o Conteúdo</a></li>
                <li><a href="#">Home</a></li>
                <li><a href="#">Artigos</a></li>
                <li><a href="#">Contacto</a></li>
            </ul>
        </nav>
    </header>

    <main id="conteudo-principal">
        <section>
            <h2>O Que é Acessibilidade Web?</h2>
            <p>A acessibilidade web garante que todos, incluindo pessoas com de

            <h3>Exemplos de Boas Práticas:</h3>
            <ul>
                <li>Use <a href="#" aria-label="Saiba mais sobre texto alternat
                <li>Garanta contraste de cores suficiente.</li>
                <li>Use tags HTML semânticas.</li>
                <li>Forneça navegação por teclado.</li>
            </ul>
        </section>

        <section>
            <h2>Fundamentos de SEO</h2>
            <p>Otimização para motores de busca ajuda o seu site a ser encontra
```

```
            <h3>Dicas de SEO:</h3>
            <ol>
                <li>Use títulos e meta descrições relevantes.</li>
                <li>Crie conteúdo de alta qualidade.</li>
                <li>Garanta que o seu site é responsivo.</li>
            </ol>
        </section>

        <button role="button" aria-label="Abrir menu de opções">Opções</button>
    </main>

    <footer>
        <p>&copy; 2026 Meu Site. Todos os direitos reservados.</p>
    </footer>
</body>
</html>
```

Explicação do Exemplo

- `<meta name="description" ...>` : Fornece uma descrição concisa para os motores de busca.
- `aria-label="Navegação Principal"` no `<nav>` : Atributo ARIA (Accessible Rich Internet Applications) que fornece um rótulo descritivo para tecnologias de assistência.
- `<a href="#conteudo-principal">Ir para o Conteúdo</a>` : Um "skip link" que permite aos utilizadores de teclado saltar diretamente para o conteúdo principal, ignorando a navegação repetitiva.
- `alt="Ícone de informação"` na `<img>` : Texto alternativo que descreve a imagem para leitores de ecrã.
- `role="button"` e `aria-label="Abrir menu de opções"` no `<button>` : Mesmo que um `div` ou `span` possa ser estilizado para parecer um botão, usar a tag `<button>` nativa é melhor para acessibilidade. Se usar um elemento não-semântico, os atributos `role` e `aria-label` ajudam a comunicar a sua função.

Exercício do Dia 13

1. Revise o seu projeto de currículo (`curriculo.html`).
2. Adicione uma meta descrição relevante na secção `<head>` .
3. Verifique se todas as suas imagens têm o atributo `alt` com descrições precisas.
4. Certifique-se de que está a usar tags de cabeçalho (`<h1>` , `<h2>` , etc.) de forma hierárquica e lógica.

5. Adicione um "skip link" no topo da página que leve o utilizador diretamente para a secção principal do currículo.
6. Se tiver links ou botões que não são semanticamente óbvios, adicione `aria-label` para melhorar a acessibilidade.

Dia 14: Projeto 2: Landing Page de Produto

Conceitos

Este projeto finaliza a segunda semana, aplicando todos os conceitos de HTML e CSS aprendidos, com foco especial em layouts modernos (Flexbox/Grid) e design responsivo. O objetivo é criar uma **Landing Page** para um produto fictício.

Uma landing page é uma página web autónoma, criada especificamente para uma campanha de marketing ou publicidade. É onde um visitante "aterrissa" depois de clicar num anúncio ou link. O seu principal objetivo é a **conversão** (ex: vender um produto, recolher um email, fazer um download).

Elementos comuns de uma Landing Page:

- **Cabeçalho:** Com logótipo e, opcionalmente, um menu de navegação simples.
- **Hero Section:** A primeira secção visível, com um título chamativo, uma breve descrição do produto, uma imagem/vídeo e uma "Call to Action" (CTA) proeminente.
- **Secção de Funcionalidades/Benefícios:** Lista os principais recursos e como o produto resolve os problemas do utilizador.
- **Testemunhos:** Citações de clientes satisfeitos para construir confiança.
- **Preços (opcional):** Planos e opções de compra.
- **Outra CTA:** Reforçar o apelo à ação.
- **Rodapé:** Informações de contacto, links para políticas de privacidade, termos de serviço.

Requisitos Técnicos

- Utilize HTML semântico para estruturar a página.
- Use Flexbox e/ou CSS Grid para criar o layout principal da página e alinhar elementos.
- Implemente um design **totalmente responsivo** usando Media Queries, garantindo que a página se adapta bem a telemóveis, tablets e desktops.
- Aplique transições e animações subtis para melhorar a experiência do utilizador.
- Use variáveis CSS para gerir cores e espaçamentos de forma eficiente.

- Considere as boas práticas de acessibilidade e SEO básicas.

Exercício do Dia 14

Crie um novo arquivo `landing_page.html` e `landing_page.css`.

1. Escolha um produto fictício (ex: uma aplicação de produtividade, um serviço de streaming, um gadget inovador).
2. Desenhe e implemente a estrutura HTML da landing page, incluindo todas as secções mencionadas acima.
3. Estilize a página com CSS, criando um design moderno e limpo.
4. Certifique-se de que a página é totalmente responsiva, ajustando o layout e os tamanhos de fonte conforme o tamanho do ecrã.
5. Adicione pelo menos um botão de "Call to Action" com um efeito de `hover` animado.

Dica: Foque-se na clareza da mensagem e na facilidade de uso. Uma boa landing page é simples e direciona o utilizador para a ação desejada.

Semana 3: Lógica de Programação com JavaScript (A Interatividade da Web)

Dia 15: Variáveis, Tipos de Dados e Operadores

Conceitos

O **JavaScript** é uma linguagem de programação que permite adicionar interatividade às suas páginas web. É uma das três tecnologias fundamentais do desenvolvimento web, juntamente com HTML e CSS.

Variáveis são "contentores" para armazenar dados. Em JavaScript, pode declarar variáveis usando `var`, `let` ou `const`.

- `var`: Declara uma variável com escopo de função. É a forma mais antiga e tem algumas peculiaridades.
- `let`: Declara uma variável com escopo de bloco. É a forma preferida para variáveis que podem ser reatribuídas.
- `const`: Declara uma constante com escopo de bloco. O seu valor não pode ser reatribuído após a inicialização. É a forma preferida para valores que não devem mudar.

Tipos de Dados em JavaScript:

- **Primitivos:**

- `String` : Texto (ex: `'Olá Mundo'`).
- `Number` : Números inteiros e de ponto flutuante (ex: `10` , `3.14`).
- `Boolean` : Valores verdadeiros ou falsos (`true` , `false`).
- `Undefined` : Uma variável que foi declarada, mas ainda não recebeu um valor.
- `Null` : Representa a ausência intencional de qualquer valor de objeto.
- `Symbol` (ES6): Valores únicos e imutáveis.
- `BigInt` (ES2020): Para números inteiros muito grandes.

- **Não Primitivos (Objetos):**
  - `Object` : Coleções de dados e funcionalidades (ex: `{ nome: 'João', idade: 30 }`).
  - `Array` : Listas ordenadas de valores (ex: `[1, 2, 3]`).
  - `Function` : Blocos de código reutilizáveis.

Operadores são símbolos que realizam operações em valores e variáveis.

- **Aritméticos:** `+` , `-` , `*` , `/` , `%` (módulo), `**` (exponenciação).
- **Atribuição:** `=` , `+=` , `-=` , `*=` .
- **Comparação:** `==` (igual a, compara valor), `===` (estritamente igual a, compara valor e tipo), `!=` , `!==` , `>` , `<` , `>=` , `<=` .
- **Lógicos:** `&&` (AND), `||` (OR), `!` (NOT).
- **Incremento/Decremento:** `++` , `--` .

Exemplo de Código: `script.js`

Crie um novo arquivo `script.js` na mesma pasta e adicione o seguinte código. Pode abri-lo no navegador e ver o resultado na consola (F12 -> Console).

JavaScript

```
// script.js

// 1. Declaração de Variáveis
var nome = "Alice"; // Evitar usar var em código moderno
let idade = 25;
const PI = 3.14159;

console.log("Nome:", nome);
console.log("Idade:", idade);
console.log("PI:", PI);

// Tentando reatribuir const (irá gerar um erro)
```

```
// PI = 3.14; // Descomente para ver o erro

// Reatribuindo let
idade = 26;
console.log("Nova Idade:", idade);

// 2. Tipos de Dados
let saudacao = "Olá, mundo!"; // String
let numeroInteiro = 100;      // Number
let numeroDecimal = 10.5;     // Number
let isStudent = true;         // Boolean
let indefinido;               // Undefined
let nulo = null;              // Null

console.log("Tipo de saudacao:", typeof saudacao); // string
console.log("Tipo de numeroInteiro:", typeof numeroInteiro); // number
console.log("Tipo de isStudent:", typeof isStudent); // boolean
console.log("Tipo de indefinido:", typeof indefinido); // undefined
console.log("Tipo de nulo:", typeof nulo); // object (uma peculiaridade do JS)

// Objeto e Array
let pessoa = {
    primeiroNome: "Bob",
    ultimoNome: "Silva",
    idade: 30
};

let frutas = ["Maçã", "Banana", "Laranja"];

console.log("Pessoa:", pessoa);
console.log("Frutas:", frutas);

// 3. Operadores Aritméticos
let a = 10;
let b = 5;

console.log("a + b =", a + b); // 15
console.log("a - b =", a - b); // 5
console.log("a * b =", a * b); // 50
console.log("a / b =", a / b); // 2
console.log("a % b =", a % b); // 0 (resto da divisão)
console.log("a ** 2 =", a ** 2); // 100 (a elevado a 2)

// Operadores de Atribuição
let x = 10;
x += 5; // x = x + 5; (x agora é 15)
console.log("x depois de +=:", x);
```

```
// Operadores de Comparação
let num1 = 10;
let num2 = "10";

console.log("num1 == num2:", num1 == num2);   // true (compara apenas o valor)
console.log("num1 === num2:", num1 === num2); // false (compara valor e tipo)
console.log("num1 != num2:", num1 != num2);   // false
console.log("num1 !== num2:", num1 !== num2); // true
console.log("num1 > 5:", num1 > 5);         // true

// Operadores Lógicos
let temIdade = true;
let temCarta = false;

console.log("Pode dirigir (idade E carta)?", temIdade && temCarta); // false
console.log("Pode entrar (idade OU carta)?", temIdade || temCarta); // true
console.log("Não tem carta?", !temCarta); // true

// Operadores de Incremento/Decremento
let contador = 0;
contador++; // contador agora é 1
console.log("Contador incrementado:", contador);
contador--; // contador agora é 0
console.log("Contador decrementado:", contador);
```

Para executar este script, adicione a tag `<script src="script.js"></script>` no final do `<body>` do seu `index.html` (ou crie um novo HTML para testar o JS).

Exercício do Dia 15

1. Crie um novo arquivo `calculadora_simples.js`.
2. Declare duas variáveis `numA` e `numB` e atribua-lhes valores numéricos à sua escolha.
3. Declare uma variável `operacao` e atribua-lhe uma string como "soma", "subtracao", "multiplicacao" ou "divisao".
4. Usando operadores aritméticos, calcule o resultado da operação escolhida entre `numA` e `numB`.
5. Imprima o resultado na consola, juntamente com uma mensagem descritiva (ex: "A soma de 10 e 5 é: 15").
6. Adicione um comentário ao lado de cada linha de código a explicar o que ela faz.

Dia 16: Estruturas de Controlo (If/Else, Switch)

Conceitos

As **estruturas de controlo** permitem que o seu código tome decisões e execute diferentes blocos de código com base em condições. As mais comuns são `if/else if/else` e `switch`.

`if/else if/else`:

Executa um bloco de código se uma condição for verdadeira. Pode ter múltiplas condições com `else if` e um bloco `else` para quando nenhuma das condições anteriores for verdadeira.

```JavaScript
if (condicao1) {
    // Código a ser executado se condicao1 for verdadeira
} else if (condicao2) {
    // Código a ser executado se condicao2 for verdadeira
} else {
    // Código a ser executado se nenhuma das condições for verdadeira
}
```

`switch`:

É uma alternativa ao `if/else if/else` quando se tem muitas condições baseadas no valor de uma única variável. É mais legível para múltiplos casos.

```JavaScript
switch (expressao) {
    case valor1:
        // Código a ser executado se expressao for igual a valor1
        break; // Importante para sair do switch
    case valor2:
        // Código a ser executado se expressao for igual a valor2
        break;
    default:
        // Código a ser executado se nenhum dos casos corresponder
}
```

Exemplo de Código: `condicionais.js`

```JavaScript
// condicionais.js

// Exemplo com if/else if/else
let hora = 14;
```

```
if (hora < 12) {
    console.log("Bom dia!");
} else if (hora < 18) {
    console.log("Boa tarde!");
} else {
    console.log("Boa noite!");
}

// Exemplo com switch
let diaDaSemana = "quarta";

switch (diaDaSemana) {
    case "segunda":
        console.log("Hoje é segunda-feira, início da semana.");
        break;
    case "terça":
    case "quarta": // Pode agrupar casos
        console.log("Meio da semana, foco total!");
        break;
    case "sexta":
        console.log("Sextou! Quase fim de semana.");
        break;
    default:
        console.log("Fim de semana ou dia inválido.");
}

// Operador Ternário (atalho para if/else simples)
let idadeUsuario = 18;
let podeVotar = (idadeUsuario >= 18) ? "Sim" : "Não";
console.log("Pode votar?", podeVotar);
```

Explicação do Exemplo

- O primeiro `if/else if/else` verifica a hora do dia e imprime uma saudação apropriada.
- O `switch` verifica o valor da variável `diaDaSemana` e executa o bloco de código correspondente. Note o uso de `break` para evitar que o código continue a ser executado nos casos seguintes.
- O operador ternário `(condicao) ? valorSeVerdadeiro : valorSeFalso;` é uma forma concisa de escrever um `if/else` simples.

Exercício do Dia 16

1. Crie um novo arquivo `verificador_idade.js`.
2. Declare uma variável `idade` e atribua-lhe um valor numérico.

3. Usando uma estrutura `if/else if/else`, verifique a idade e imprima na consola:
    - "É menor de idade." se a idade for inferior a 18.
    - "É adulto." se a idade for entre 18 e 64 (inclusive).
    - "É idoso." se a idade for 65 ou superior.
4. Crie uma variável `mes` e atribua-lhe um número de 1 a 12.
5. Usando uma estrutura `switch`, imprima o nome do mês correspondente (ex: "Janeiro" para 1, "Fevereiro" para 2, etc.). Se o número for inválido, imprima "Mês inválido.".

Dia 17: Loops (For, While)

Conceitos

Loops são estruturas de controlo que permitem executar um bloco de código repetidamente. São essenciais para automatizar tarefas repetitivas, como iterar sobre elementos de um array ou executar uma ação um certo número de vezes.

`for` loop:

É o loop mais comum e é ideal quando se sabe o número exato de iterações. Tem três partes: inicialização, condição e incremento/decremento.

JavaScript

```
for (inicializacao; condicao; incremento/decremento) {
    // Código a ser executado em cada iteração
}
```

`while` loop:

Executa um bloco de código enquanto uma condição especificada for verdadeira. É útil quando o número de iterações não é conhecido de antemão.

JavaScript

```
while (condicao) {
    // Código a ser executado enquanto a condição for verdadeira
    // É crucial ter algo dentro do loop que eventualmente torne a condição fal
}
```

`do...while` loop:

Semelhante ao `while`, mas garante que o bloco de código seja executado pelo menos uma vez, antes de verificar a condição.

JavaScript

```
do {
    // Código a ser executado pelo menos uma vez
} while (condicao);
```

`for...of` e `for...in` (ES6):

- `for...of`: Itera sobre os valores de objetos iteráveis (como arrays, strings, Map, Set).
- `for...in`: Itera sobre as chaves (propriedades) enumeráveis de um objeto.

Exemplo de Código: `loops.js`

JavaScript

```
// loops.js

// Exemplo com for loop
console.log("Contagem de 1 a 5 com for:");
for (let i = 1; i <= 5; i++) {
    console.log(i);
}

// Iterar sobre um array com for
const frutas = ["Maçã", "Banana", "Laranja", "Uva"];
console.log("Minhas frutas favoritas:");
for (let i = 0; i < frutas.length; i++) {
    console.log(frutas[i]);
}

// Exemplo com while loop
let contador = 0;
console.log("Contagem com while:");
while (contador < 3) {
    console.log("O contador é: " + contador);
    contador++;
}

// Exemplo com do...while loop
let j = 0;
console.log("Contagem com do...while:");
do {
    console.log("O j é: " + j);
    j++;
} while (j < 0); // Condição falsa, mas executa uma vez

// Exemplo com for...of (para arrays)
```

```
console.log("Frutas com for...of:");
for (const fruta of frutas) {
    console.log(fruta);
}

// Exemplo com for...in (para objetos)
const carro = {
    marca: "Toyota",
    modelo: "Corolla",
    ano: 2020
};

console.log("Propriedades do carro:");
for (const chave in carro) {
    console.log(chave + ": " + carro[chave]);
}
```

Explicação do Exemplo

- O `for` loop é usado para contar de 1 a 5 e para iterar sobre os elementos de um array pelo seu índice.
- O `while` loop continua a executar enquanto `contador` for menor que 3.
- O `do...while` loop executa o bloco de código pelo menos uma vez, mesmo que a condição seja falsa desde o início.
- `for...of` é uma forma mais moderna e legível de iterar sobre os valores de um array.
- `for...in` é usado para iterar sobre as chaves (propriedades) de um objeto.

Exercício do Dia 17

1. Crie um novo arquivo `tabuada.js`.
2. Use um `for` loop para gerar a tabuada de um número à sua escolha (ex: 7). Imprima cada linha da tabuada na consola (ex: "7 x 1 = 7", "7 x 2 = 14", etc., até 10).
3. Crie um array de números (ex: `[10, 20, 30, 40, 50]`).
4. Use um `for...of` loop para iterar sobre este array e imprimir cada número na consola.
5. Use um `while` loop para encontrar o primeiro número par maior que 100. Imprima esse número na consola.

Dia 18: Funções e Escopo

Conceitos

Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas ajudam a organizar o código, torná-lo mais modular e evitar a repetição de código (princípio DRY - Don't Repeat Yourself).

Uma função pode receber **parâmetros** (valores de entrada) e pode **retornar** um valor.

Existem várias formas de declarar funções em JavaScript:

1. **Declaração de Função (Function Declaration):**

   JavaScript
   ```
   function nomeDaFuncao(parametro1, parametro2) {
       // Código da função
       return resultado;
   }
   ```

2. **Expressão de Função (Function Expression):**

   JavaScript
   ```
   const nomeDaFuncao = function(parametro1, parametro2) {
       // Código da função
       return resultado;
   };
   ```

3. **Arrow Functions (ES6):** Uma sintaxe mais concisa, especialmente útil para funções anónimas ou de uma única linha.

   JavaScript
   ```
   const nomeDaFuncao = (parametro1, parametro2) => {
       // Código da função
       return resultado;
   };

   // Para funções de uma única linha, pode ser ainda mais concisa:
   const somar = (a, b) => a + b;
   ```

Escopo (Scope) refere-se à visibilidade das variáveis no seu código. Em JavaScript, existem principalmente dois tipos de escopo:

- **Escopo Global:** Variáveis declaradas fora de qualquer função ou bloco. Podem ser acedidas de qualquer lugar no código.
- **Escopo de Função (Function Scope):** Variáveis declaradas dentro de uma função. Só podem ser acedidas dentro dessa função.

- **Escopo de Bloco (Block Scope - com `let` e `const`):** Variáveis declaradas dentro de um bloco (`{}`) (ex: dentro de um `if`, `for`, `while`). Só podem ser acedidas dentro desse bloco.

Exemplo de Código: `funcoes.js`

JavaScript

```
// funcoes.js

// 1. Declaração de Função
function saudar(nome) {
    return "Olá, " + nome + "!";
}

console.log(saudar("Maria")); // Olá, Maria!

// 2. Expressão de Função
const multiplicar = function(a, b) {
    return a * b;
};

console.log(multiplicar(5, 3)); // 15

// 3. Arrow Function
const dividir = (a, b) => a / b;
console.log(dividir(10, 2)); // 5

const apresentar = (nome, idade) => {
    const mensagem = `O meu nome é ${nome} e tenho ${idade} anos.`;
    return mensagem;
};
console.log(apresentar("Pedro", 30));

// Exemplo de Escopo
let variavelGlobal = "Sou global";

function mostrarEscopo() {
    let variavelFuncao = "Sou da função";
    console.log(variavelGlobal); // Acessível
    console.log(variavelFuncao); // Acessível

    if (true) {
        let variavelBloco = "Sou do bloco";
        console.log(variavelFuncao); // Acessível
        console.log(variavelBloco); // Acessível
    }
```

```
    // console.log(variavelBloco); // Erro: variavelBloco não está definida aqu
}

mostrarEscopo();

console.log(variavelGlobal); // Acessível
// console.log(variavelFuncao); // Erro: variavelFuncao não está definida aqui
```

Explicação do Exemplo

- Demonstra as três formas de declarar funções e como chamá-las.
- O exemplo de escopo mostra que `variavelGlobal` é acessível em qualquer lugar, `variavelFuncao` apenas dentro de `mostrarEscopo()`, e `variavelBloco` apenas dentro do bloco `if`.
- O uso de `let` e `const` para variáveis de bloco é uma prática moderna que ajuda a evitar problemas de escopo que eram comuns com `var`.

Exercício do Dia 18

1. Crie um novo arquivo `funcoes_exercicio.js`.
2. Escreva uma função chamada `calcularAreaRetangulo` que receba dois parâmetros: `largura` e `altura`. A função deve retornar a área do retângulo.
3. Escreva uma função chamada `verificarParImpar` que receba um número como parâmetro. A função deve imprimir na consola se o número é par ou ímpar (use `if/else` e o operador `%`).
4. Escreva uma arrow function chamada `saudacaoPersonalizada` que receba um nome e retorne uma string como "Olá, [Nome]! Bem-vindo(a)!".
5. Chame todas as funções que criou com valores de teste e imprima os resultados na consola.

Dia 19: Arrays e Objetos

Conceitos

Arrays são listas ordenadas de valores. Cada valor num array é chamado de elemento, e cada elemento tem um índice numérico (começando em 0).

JavaScript

```
const meuArray = [elemento0, elemento1, elemento2];
```

Operações comuns com Arrays:

- Aceder elementos: `meuArray[indice]`
- Adicionar elementos: `push()` (final), `unshift()` (início)
- Remover elementos: `pop()` (final), `shift()` (início)
- Encontrar o tamanho: `meuArray.length`
- Iterar: `for` loop, `forEach()`, `map()`, `filter()`, `reduce()`.

Objetos são coleções de propriedades, onde cada propriedade tem um nome (chave) e um valor. São usados para representar entidades mais complexas, como uma pessoa, um carro, etc.

JavaScript

```
const meuObjeto = {
    chave1: valor1,
    chave2: valor2
};
```

Operações comuns com Objetos:

- Aceder propriedades: `meuObjeto.chave` ou `meuObjeto['chave']`
- Adicionar/Modificar propriedades: `meuObjeto.novaChave = novoValor`
- Remover propriedades: `delete meuObjeto.chave`
- Iterar: `for...in`, `Object.keys()`, `Object.values()`, `Object.entries()`.

Exemplo de Código: `arrays_objetos.js`

JavaScript

```
// arrays_objetos.js

// Exemplo de Array
const cores = ["Vermelho", "Verde", "Azul"];
console.log("Array de cores:", cores);

// Aceder elementos
console.log("Primeira cor:", cores[0]); // Vermelho
console.log("Última cor:", cores[cores.length - 1]); // Azul

// Adicionar elementos
cores.push("Amarelo"); // Adiciona ao final
console.log("Cores depois de push:", cores);
cores.unshift("Roxo"); // Adiciona ao início
```

```
console.log("Cores depois de unshift:", cores);

// Remover elementos
const ultimaCor = cores.pop(); // Remove do final
console.log("Cor removida (pop):",
 ultimaCor);
console.log("Cores depois de pop:", cores);
const primeiraCor = cores.shift(); // Remove do início
console.log("Cor removida (shift):",
 primeiraCor);
console.log("Cores depois de shift:", cores);

// Iterar com forEach
console.log("Iterando cores com forEach:");
cores.forEach(function(cor) {
    console.log(cor);
});

// Exemplo de Objeto
const utilizador = {
    nome: "Ana",
    sobrenome: "Santos",
    idade: 28,
    email: "ana.santos@example.com",
    hobbies: ["leitura", "caminhada", "cozinhar"],
    endereco: {
        rua: "Rua Principal",
        numero: 123,
        cidade: "Lisboa"
    }
};

console.log("Objeto utilizador:", utilizador);

// Aceder propriedades
console.log("Nome do utilizador:", utilizador.nome); // Ana
console.log("Cidade do utilizador:", utilizador.endereco.cidade); // Lisboa
console.log("Primeiro hobby:", utilizador.hobbies[0]); // leitura

// Modificar propriedade
utilizador.idade = 29;
console.log("Nova idade:", utilizador.idade);

// Adicionar nova propriedade
utilizador.profissao = "Designer";
console.log("Profissão:", utilizador.profissao);

// Remover propriedade
```

```
delete utilizador.email;
console.log("Utilizador sem email:", utilizador);

// Iterar sobre chaves de objeto com for...in
console.log("Propriedades do utilizador:");
for (const chave in utilizador) {
    console.log(`${chave}: ${utilizador[chave]}`);
}
```

Explicação do Exemplo

- Demonstra a criação de arrays e objetos, e como aceder, adicionar, modificar e remover elementos/propriedades.
- `push()` e `unshift()` adicionam elementos, enquanto `pop()` e `shift()` removem.
- `forEach()` é um método de array para iterar sobre cada elemento.
- Objetos podem conter outros objetos e arrays, permitindo estruturas de dados complexas.
- Aceder propriedades de objeto pode ser feito com notação de ponto (`.`) ou notação de colchetes (`[]`).

Exercício do Dia 19

1. Crie um novo arquivo `gerenciador_tarefas.js`.
2. Crie um array chamado `tarefas` que contenha 3 objetos. Cada objeto deve representar uma tarefa e ter as seguintes propriedades:
   - `id` (Number)
   - `descricao` (String)
   - `concluida` (Boolean)
3. Adicione uma nova tarefa ao array `tarefas`.
4. Altere o estado `concluida` de uma das tarefas existentes para `true`.
5. Use um `for` loop ou `forEach` para iterar sobre o array `tarefas` e imprimir a `descricao` de cada tarefa na consola, indicando se está concluída ou não.

Dia 20: ES6+ (Arrow Functions, Destructuring, Spread/Rest)

Conceitos

O **ECMAScript 2015 (ES6)** e versões posteriores introduziram muitas funcionalidades novas e melhorias ao JavaScript, tornando-o mais moderno, conciso e poderoso. Algumas

das mais importantes incluem:

- **Arrow Functions:** Já abordadas no Dia 18, oferecem uma sintaxe mais curta para escrever expressões de função e não têm o seu próprio `this` (herdam do contexto pai).
- **Destructuring Assignment (Desestruturação):** Permite extrair valores de arrays ou propriedades de objetos para variáveis distintas de forma mais concisa.

  JavaScript
  ```
  // Desestruturação de Array
  const numeros = [1, 2, 3];
  const [primeiro, segundo, terceiro] = numeros;
  console.log(primeiro); // 1

  // Desestruturação de Objeto
  const pessoa = { nome: "João", idade: 30 };
  const { nome, idade } = pessoa;
  console.log(nome); // João
  ```

- **Spread Operator (`...`):** Permite expandir um iterável (como um array ou string) em elementos individuais, ou expandir um objeto em pares chave-valor.

  JavaScript
  ```
  // Com Arrays
  const arr1 = [1, 2];
  const arr2 = [3, 4];
  const combinado = [...arr1, ...arr2]; // [1, 2, 3, 4]

  // Com Objetos
  const obj1 = { a: 1, b: 2 };
  const obj2 = { c: 3, d: 4 };
  const objCombinado = { ...obj1, ...obj2 }; // { a: 1, b: 2, c: 3, d: 4 }
  ```

- **Rest Parameters (`...`):** Permite representar um número indefinido de argumentos como um array.

  JavaScript
  ```
  function somarTudo(...numeros) {
      return numeros.reduce((total, num) => total + num, 0);
  }
  console.log(somarTudo(1, 2, 3, 4)); // 10
  ```

- **Template Literals (Template Strings):** Permitem incorporar expressões e strings multilinhas de forma mais fácil usando `` ``` `` (backticks).

  JavaScript

  ```
  const nome = "Mundo";
  const saudacao = `Olá, ${nome}!`; // Olá, Mundo!
  ```

Exemplo de Código: `es6_features.js`

JavaScript

```
// es6_features.js

// Template Literals
const produto = "Laptop";
const preco = 1200;
const mensagem = `O ${produto} custa ${preco} euros.`;
console.log(mensagem);

// Desestruturação de Objeto
const utilizador = {
    primeiroNome: "Carla",
    ultimoNome: "Pereira",
    cidade: "Porto",
    idade: 35
};

const { primeiroNome, cidade } = utilizador;
console.log(`Nome: ${primeiroNome}, Cidade: ${cidade}`);

// Desestruturação de Array
const coresRGB = [255, 0, 128];
const [red, green, blue] = coresRGB;
console.log(`Red: ${red}, Green: ${green}, Blue: ${blue}`);

// Spread Operator com Arrays
const lista1 = ["a", "b"];
const lista2 = ["c", "d"];
const listaCompleta = [...lista1, "x", ...lista2];
console.log("Lista Completa:", listaCompleta);

// Spread Operator com Objetos
const configPadrao = { tema: "claro", notificacoes: true };
const configUsuario = { notificacoes: false, idioma: "pt" };
const configuracaoFinal = { ...configPadrao, ...configUsuario };
```

```
console.log("Configuração Final:", configuracaoFinal);

// Rest Parameters
function logArgumentos(primeiroArg, ...restantes) {
    console.log("Primeiro argumento:", primeiroArg);
    console.log("Argumentos restantes:", restantes);
}

logArgumentos(1, 2, 3, 4, 5);
```

Explicação do Exemplo

- **Template Literals:** Permitem criar strings complexas de forma mais limpa, evitando concatenação com `+`.
- **Desestruturação:** Simplifica a extração de dados de arrays e objetos, tornando o código mais legível.
- **Spread Operator:** Útil para combinar arrays/objetos ou para passar elementos de um array como argumentos individuais para uma função.
- **Rest Parameters:** Permite que uma função aceite um número variável de argumentos, agrupando-os num array.

Exercício do Dia 20

1. Crie um novo arquivo `es6_exercicio.js`.
2. Crie um objeto `produto` com as propriedades `nome`, `preco` e `disponivel`.
3. Use a desestruturação de objeto para extrair `nome` e `preco` para variáveis separadas e imprima-as usando um template literal.
4. Crie dois arrays de números, `pares` e `impares`.
5. Use o spread operator para criar um novo array `todosNumeros` que combine `pares` e `impares`.
6. Escreva uma função `media` que aceite um número variável de argumentos (usando rest parameters) e retorne a média desses números. Teste a função com diferentes conjuntos de números.

Dia 21: Projeto 3: Calculadora de Impostos

Conceitos

Este projeto visa aplicar os conceitos de JavaScript aprendidos na Semana 3: variáveis, tipos de dados, operadores, estruturas de controlo (if/else, switch) e funções. O objetivo é

criar uma calculadora simples que calcule impostos ou taxas com base num valor de entrada e algumas condições.

O projeto deve incluir:

- Um input HTML para o utilizador inserir um valor base (ex: salário, preço de um produto).
- Botões ou seletores para escolher diferentes cenários de cálculo (ex: diferentes taxas de imposto, diferentes tipos de produto).
- Um botão para acionar o cálculo.
- Uma área para exibir o resultado do cálculo.

Requisitos Técnicos

- Utilize HTML para a estrutura básica da calculadora (input, botões, área de resultado).
- Use CSS para estilizar a calculadora, tornando-a funcional e agradável visualmente.
- No JavaScript:
  - Declare variáveis para armazenar o valor de entrada, a taxa de imposto, o resultado, etc.
  - Use funções para encapsular a lógica de cálculo.
  - Implemente estruturas `if/else` ou `switch` para aplicar diferentes taxas de imposto com base na escolha do utilizador ou no valor de entrada.
  - Garanta que os inputs são tratados corretamente (ex: converter strings para números).
  - Exiba o resultado de forma clara na página HTML.

Exercício do Dia 21

Crie um novo arquivo `calculadora_impostos.html`, `calculadora_impostos.css` e `calculadora_impostos.js`.

1. **HTML:** Crie um formulário simples com:
   - Um `input` do tipo `number` para o "Valor Base".
   - Um `select` com opções para "Tipo de Imposto" (ex: "IVA 23%", "IVA 13%", "IVA 6%").
   - Um botão para "Calcular".
   - Um `p` ou `span` com um `id` para exibir o "Resultado".
2. **CSS:** Estilize a calculadora para que tenha um bom aspeto.
3. **JavaScript:**

- Obtenha referências aos elementos HTML usando `document.getElementById()`.
- Adicione um `event listener` ao botão de calcular.
- Dentro da função do `event listener`:
  - Obtenha o valor do input e o tipo de imposto selecionado.
  - Converta o valor do input para um número.
  - Use uma estrutura `switch` (ou `if/else if`) para aplicar a taxa de imposto correta com base na seleção.
  - Calcule o valor final.
  - Exiba o resultado na área de resultado HTML.

Dica: Comece com a lógica JavaScript na consola para garantir que os cálculos estão corretos antes de tentar manipular o HTML.

Semana 4: DOM, APIs e Projeto Final (Tornando a Web Interativa e Dinâmica)

Dia 22: Manipulação do DOM (Seleção e Estilo)

Conceitos

O **DOM (Document Object Model)** é uma interface de programação para documentos HTML e XML. Ele representa a estrutura de uma página web como uma árvore de objetos, onde cada nó é um elemento HTML, atributo ou texto. O JavaScript pode usar o DOM para aceder e manipular o conteúdo, estrutura e estilo de uma página web.

Seleção de Elementos:

Para manipular um elemento, primeiro é preciso selecioná-lo. Métodos comuns:

- `document.getElementById("id")`: Seleciona um elemento pelo seu atributo `id` (retorna um único elemento).
- `document.getElementsByClassName("classe")`: Seleciona elementos pela sua `class` (retorna uma HTMLCollection).
- `document.getElementsByTagName("tag")`: Seleciona elementos pelo nome da tag (retorna uma HTMLCollection).
- `document.querySelector("seletor_css")`: Seleciona o **primeiro** elemento que corresponde a um seletor CSS (ex: `"#meuId"`, `".minhaClasse"`, `"p"`).

- `document.querySelectorAll("seletor_css")` : Seleciona **todos** os elementos que correspondem a um seletor CSS (retorna uma NodeList).

Manipulação de Conteúdo:

- `elemento.textContent` : Obtém ou define o conteúdo de texto de um elemento.
- `elemento.innerHTML` : Obtém ou define o conteúdo HTML (incluindo tags) de um elemento.

Manipulação de Estilo:

- `elemento.style.propriedadeCSS` : Define estilos inline para um elemento (ex: `elemento.style.color = "red";`).
- `elemento.classList.add("classe")` : Adiciona uma classe CSS a um elemento.
- `elemento.classList.remove("classe")` : Remove uma classe CSS de um elemento.
- `elemento.classList.toggle("classe")` : Alterna a presença de uma classe CSS.

Exemplo de Código: `dom_manipulacao.html` e `dom_manipulacao.js`

Crie um novo arquivo `dom_manipulacao.html` :

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Manipulação do DOM</title>
    <style>
        .destaque {
            background-color: yellow;
            font-weight: bold;
        }
        .vermelho {
            color: red;
        }
        .azul {
            color: blue;
        }
        .caixa {
            width: 100px;
            height: 100px;
            border: 2px solid black;
            margin: 10px;
            display: inline-block;
```

```
            text-align: center;
            line-height: 100px;
        }
    </style>
</head>
<body>
    <h1 id="titulo">Olá, DOM!</h1>
    <p class="paragrafo">Este é um parágrafo de exemplo.</p>
    <p class="paragrafo">Este é outro parágrafo.</p>

    <button id="btnMudarTexto">Mudar Título</button>
    <button id="btnAdicionarClasse">Destacar Parágrafos</button>
    <button id="btnAlternarCor">Alternar Cor da Caixa</button>

    <div id="minhaCaixa" class="caixa">Caixa</div>

    <script src="dom_manipulacao.js"></script>
</body>
</html>
```

E crie `dom_manipulacao.js` :

JavaScript

```
// dom_manipulacao.js

// Selecionar elementos
const titulo = document.getElementById("titulo");
const paragrafos = document.getElementsByClassName("paragrafo"); // HTMLCollect
const primeiroParagrafo = document.querySelector(".paragrafo"); // Primeiro ele
const todosParagrafos = document.querySelectorAll(".paragrafo"); // NodeList
const btnMudarTexto = document.getElementById("btnMudarTexto");
const btnAdicionarClasse = document.getElementById("btnAdicionarClasse");
const btnAlternarCor = document.getElementById("btnAlternarCor");
const minhaCaixa = document.getElementById("minhaCaixa");

// Manipular conteúdo
titulo.textContent = "DOM Manipulado!";

// Manipular estilo inline
primeiroParagrafo.style.backgroundColor = "lightblue";

// Manipular classes CSS
btnAdicionarClasse.addEventListener("click", () => {
    // Adiciona a classe 'destaque' a todos os parágrafos
    for (let i = 0; i < paragrafos.length; i++) {
        paragrafos[i].classList.add("destaque");
    }
```

```
});

let isBlue = false;
btnAlternarCor.addEventListener("click", () => {
    if (isBlue) {
        minhaCaixa.classList.remove("azul");
        minhaCaixa.classList.add("vermelho");
    } else {
        minhaCaixa.classList.remove("vermelho");
        minhaCaixa.classList.add("azul");
    }
    isBlue = !isBlue; // Inverte o estado
});

// Outra forma de alternar classes
// btnAlternarCor.addEventListener("click", () => {
//     minhaCaixa.classList.toggle("azul");
//     minhaCaixa.classList.toggle("vermelho");
// });
```

Explicação do Exemplo

- Mostra diferentes métodos para selecionar elementos HTML.
- `textContent` é usado para alterar o texto de um elemento.
- `style.backgroundColor` demonstra como aplicar estilos CSS diretamente via JavaScript.
- `classList.add()`, `remove()`, `toggle()` são métodos poderosos para gerir classes CSS, permitindo mudar a aparência dos elementos de forma dinâmica.
- `addEventListener()` é usado para reagir a eventos do utilizador (neste caso, cliques nos botões).

Exercício do Dia 22

1. Crie um novo arquivo `lista_dinamica.html` e `lista_dinamica.js`.
2. No HTML, crie:
   - Um `input` de texto.
   - Um botão "Adicionar Item".
   - Uma lista não ordenada (`<ul>`) vazia com um `id`.
3. No JavaScript:
   - Obtenha referências ao input, botão e lista.
   - Adicione um `event listener` ao botão "Adicionar Item".

- Quando o botão for clicado:
  - Obtenha o valor do input.
  - Crie um novo elemento `<li>`.
  - Defina o `textContent` do `<li>` para o valor do input.
  - Adicione o novo `<li>` à lista `<ul>`.
  - Limpe o valor do input.
4. Adicione um pouco de CSS para estilizar a lista e os itens.

Dia 23: Event Listeners (Clicks, Inputs)

Conceitos

Event Listeners (Ouvintes de Eventos) são funções em JavaScript que "escutam" eventos que ocorrem numa página web (ex: clique do rato, pressionar de tecla, carregamento da página, envio de formulário). Quando um evento é detetado, a função associada é executada.

A forma mais comum de adicionar um event listener é usando o método `addEventListener()`:

```javascript
elemento.addEventListener("nomeDoEvento", funcaoManipuladora);
```

- `elemento`: O elemento HTML ao qual o event listener será anexado.
- `"nomeDoEvento"`: Uma string que especifica o tipo de evento a ser escutado (ex: `"click"`, `"mouseover"`, `"keydown"`, `"submit"`, `"load"`).
- `funcaoManipuladora`: A função que será executada quando o evento ocorrer. Esta função recebe um objeto `event` como argumento, que contém informações sobre o evento.

Eventos comuns:

- **Eventos de Rato:** `click`, `dblclick`, `mouseover`, `mouseout`, `mousemove`.
- **Eventos de Teclado:** `keydown`, `keyup`, `keypress`.
- **Eventos de Formulário:** `submit`, `change`, `focus`, `blur`.
- **Eventos de Documento/Janela:** `load`, `resize`, `scroll`.

Exemplo de Código: `eventos.html` e `eventos.js`

Crie um novo arquivo `eventos.html`:

HTML

```html
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Event Listeners</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }
        #meuBotao {
            padding: 10px 20px;
            font-size: 1.2em;
            cursor: pointer;
            margin-bottom: 20px;
        }
        #campoTexto {
            padding: 10px;
            font-size: 1em;
            margin-bottom: 20px;
        }
        #resultado {
            font-size: 1.1em;
            color: blue;
        }
    </style>
</head>
<body>
    <h1>Event Listeners em Ação</h1>

    <button id="meuBotao">Clique-me!</button>
    <p id="mensagemClique"></p>

    <input type="text" id="campoTexto" placeholder="Digite algo...">
    <p id="mensagemInput"></p>

    <form id="meuFormulario">
        <input type="text" placeholder="Nome">
        <button type="submit">Enviar</button>
    </form>
    <p id="mensagemFormulario"></p>

    <script src="eventos.js"></script>
```

```
</body>
</html>
```

E crie `eventos.js`:

JavaScript

```
// eventos.js

const meuBotao = document.getElementById("meuBotao");
const mensagemClique = document.getElementById("mensagemClique");
const campoTexto = document.getElementById("campoTexto");
const mensagemInput = document.getElementById("mensagemInput");
const meuFormulario = document.getElementById("meuFormulario");
const mensagemFormulario = document.getElementById("mensagemFormulario");

// Evento de clique no botão
meuBotao.addEventListener("click", function() {
    mensagemClique.textContent = "Botão clicado!";
    console.log("Botão clicado!");
});

// Evento de input no campo de texto
campoTexto.addEventListener("input", function(event) {
    mensagemInput.textContent = `Você digitou: ${event.target.value}`;
});

// Evento de envio de formulário
meuFormulario.addEventListener("submit", function(event) {
    event.preventDefault(); // Previne o comportamento padrão de recarregar a p
    const nomeInput = meuFormulario.querySelector("input[type='text']");
    mensagemFormulario.textContent = `Formulário enviado! Nome: ${nomeInput.val
    console.log("Formulário enviado!");
    nomeInput.value = ''; // Limpa o campo
});

// Evento de mouseover (passar o rato por cima)
meuBotao.addEventListener("mouseover", function() {
    meuBotao.style.backgroundColor = "lightgreen";
});

// Evento de mouseout (tirar o rato de cima)
meuBotao.addEventListener("mouseout", function() {
    meuBotao.style.backgroundColor = ""; // Volta ao estilo original
});
```

Explicação do Exemplo

- O `click` event listener no botão executa uma função quando o botão é clicado.
- O `input` event listener no campo de texto reage a cada alteração no valor do input, mostrando o texto digitado em tempo real.
- O `submit` event listener no formulário demonstra como intercetar o envio de um formulário. `event.preventDefault()` é crucial para impedir que a página seja recarregada, o que é o comportamento padrão de um formulário HTML.
- `event.target.value` acede o valor do elemento que disparou o evento.
- `mouseover` e `mouseout` são usados para adicionar e remover estilos quando o rato entra e sai de um elemento.

Exercício do Dia 23

1. Crie um novo arquivo `contador_caracteres.html` e `contador_caracteres.js`.
2. No HTML, crie:
   - Uma `textarea` com um `id`.
   - Um `p` com um `id` para exibir a contagem de caracteres.
3. No JavaScript:
   - Obtenha referências à `textarea` e ao parágrafo de contagem.
   - Adicione um `event listener` do tipo `input` à `textarea`.
   - Dentro da função do `event listener`:
     - Obtenha o texto da `textarea`.
     - Conte o número de caracteres.
     - Atualize o `textContent` do parágrafo de contagem com a mensagem "Caracteres: [número]".
4. Adicione um pouco de CSS para estilizar a `textarea` e o contador.

Dia 24: JSON e Introdução a APIs (Fetch)

Conceitos

JSON (JavaScript Object Notation) é um formato leve de troca de dados. É fácil para humanos lerem e escreverem, e fácil para máquinas analisarem e gerarem. É amplamente utilizado para enviar dados entre um servidor e um cliente web (navegador).

Um objeto JSON é uma coleção de pares nome/valor, e um array JSON é uma lista ordenada de valores. Os valores podem ser strings, números, booleanos, null, objetos ou arrays.

JSON

```
{
    "nome": "Carlos",
    "idade": 40,
    "isEstudante": false,
    "hobbies": ["jogar", "ler"],
    "endereco": {
        "rua": "Rua A",
        "numero": 10
    }
}
```

APIs (Application Programming Interfaces) são conjuntos de regras e protocolos que permitem que diferentes aplicações de software comuniquem entre si. No contexto web, uma API RESTful permite que a sua aplicação frontend (o navegador) solicite e receba dados de um servidor.

O **Fetch API** é uma interface moderna e poderosa para fazer requisições de rede (como obter dados de uma API) em JavaScript. Ele retorna uma `Promise`, o que o torna ideal para trabalhar com operações assíncronas.

Sintaxe básica do Fetch:

JavaScript

```
fetch("url_da_api")
    .then(response => response.json()) // Converte a resposta para JSON
    .then(data => {
        // Processa os dados recebidos
        console.log(data);
    })
    .catch(error => {
        // Lida com erros
        console.error("Erro ao buscar dados:", error);
    });
```

- `fetch()`: Inicia a requisição.
- `.then()`: É executado quando a Promise é resolvida com sucesso. O primeiro `.then()` geralmente converte a resposta para JSON.
- `.catch()`: É executado se houver um erro na requisição.

Exemplo de Código: `api_data.html` e `api_data.js`

Crie um novo arquivo `api_data.html`:

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Consumindo APIs</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }
        #btnCarregarDados {
            padding: 10px 20px;
            font-size: 1.2em;
            cursor: pointer;
            margin-bottom: 20px;
        }
        #dadosApi {
            margin-top: 20px;
            border: 1px solid #ccc;
            padding: 20px;
            max-width: 600px;
            margin-left: auto;
            margin-right: auto;
            text-align: left;
        }
    </style>
</head>
<body>
    <h1>Dados de uma API Externa</h1>

    <button id="btnCarregarDados">Carregar Dados de Utilizadores</button>
    <div id="dadosApi"></div>

    <script src="api_data.js"></script>
</body>
</html>
```

E crie `api_data.js`:

```javascript
// api_data.js

const btnCarregarDados = document.getElementById("btnCarregarDados");
const dadosApiDiv = document.getElementById("dadosApi");

btnCarregarDados.addEventListener("click", () => {
    dadosApiDiv.textContent = "A carregar dados...";

    // Exemplo de API pública: JSONPlaceholder (para dados de teste)
    fetch("https://jsonplaceholder.typicode.com/users" )
        .then(response => {
            if (!response.ok) {
                throw new Error(`Erro HTTP! Status: ${response.status}`);
            }
            return response.json(); // Converte a resposta para JSON
        })
        .then(data => {
            // Processa os dados recebidos
            dadosApiDiv.innerHTML = "<h2>Lista de Utilizadores:</h2>";
            const ul = document.createElement("ul");
            data.forEach(user => {
                const li = document.createElement("li");
                li.textContent = `Nome: ${user.name}, Email: ${user.email}`;
                ul.appendChild(li);
            });
            dadosApiDiv.appendChild(ul);
        })
        .catch(error => {
            // Lida com erros
            dadosApiDiv.textContent = `Erro ao carregar dados: ${error.message}
            console.error("Erro na requisição Fetch:", error);
        });
});
```

Explicação do Exemplo

- O `fetch()` faz uma requisição GET para a API de utilizadores do JSONPlaceholder.
- O primeiro `.then()` verifica se a resposta HTTP foi bem-sucedida (`response.ok`) e, em seguida, usa `response.json()` para analisar o corpo da resposta como JSON.
- O segundo `.then()` recebe os dados JSON (um array de objetos de utilizador) e os exibe na página, criando elementos `<li>` dinamicamente.
- O `.catch()` captura quaisquer erros que possam ocorrer durante a requisição ou o processamento da resposta.

Exercício do Dia 24

1. Crie um novo arquivo `clima.html` e `clima.js`.
2. No HTML, crie:
   - Um `input` de texto para o utilizador digitar o nome de uma cidade.
   - Um botão "Obter Clima".
   - Um `div` com um `id` para exibir as informações do clima.
3. No JavaScript:
   - Obtenha referências ao input, botão e div de resultado.
   - Adicione um `event listener` ao botão.
   - Dentro da função do `event listener`:
     - Obtenha o nome da cidade do input.
     - Use o `Fetch API` para consultar uma API de clima pública (ex: OpenWeatherMap API - precisará de uma chave API gratuita, ou pode usar uma API mais simples que não exija chave para fins de aprendizagem, como WeatherAPI.com Free). Se usar OpenWeatherMap, o URL seria algo como `https://api.openweathermap.org/data/2.5/weather?q=${cidade}&appid=SUA_CHAVE_API&units=metric`.
     - Exiba o nome da cidade, temperatura e uma breve descrição do clima na div de resultado.
     - Lide com possíveis erros (cidade não encontrada, erro de rede).

Dica: Para o OpenWeatherMap, registe-se para obter uma chave API gratuita. Para simplificar, pode usar uma chave API de exemplo ou uma API que não exija autenticação para este exercício.

Dia 25: Armazenamento Local (LocalStorage)

Conceitos

O **LocalStorage** é uma forma de armazenar dados no navegador do utilizador de forma persistente, ou seja, os dados permanecem mesmo depois que o navegador é fechado e reaberto. É útil para guardar preferências do utilizador, tokens de autenticação ou dados offline.

O LocalStorage armazena dados como pares chave-valor, onde tanto a chave quanto o valor são strings. Se precisar de armazenar objetos ou arrays, terá que convertê-los para strings JSON antes de guardar e converter de volta ao ler.

Métodos principais do LocalStorage:

- `localStorage.setItem("chave", "valor")` : Armazena um par chave-valor.
- `localStorage.getItem("chave")` : Recupera o valor associado a uma chave.
- `localStorage.removeItem("chave")` : Remove um par chave-valor.
- `localStorage.clear()` : Remove todos os pares chave-valor do LocalStorage.

Sessão Storage é semelhante ao LocalStorage, mas os dados são apagados quando a sessão do navegador termina (quando a aba ou janela é fechada).

Exemplo de Código: `local_storage.html` e `local_storage.js`

Crie um novo arquivo `local_storage.html` :

HTML

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LocalStorage</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }
        input[type="text"] {
            padding: 10px;
            font-size: 1em;
            margin-bottom: 10px;
        }
        button {
            padding: 10px 15px;
            font-size: 1em;
            cursor: pointer;
            margin: 5px;
        }
        #mensagemSalva {
            margin-top: 20px;
            font-weight: bold;
            color: green;
        }
        #preferenciasUsuario {
            margin-top: 20px;
            border: 1px solid #ccc;
```

```
            padding: 15px;
            max-width: 400px;
            margin-left: auto;
            margin-right: auto;
            text-align: left;
        }
    </style>
</head>
<body>
    <h1>Armazenamento Local</h1>

    <h2>Guardar uma Mensagem</h2>
    <input type="text" id="inputMensagem" placeholder="Digite uma mensagem">
    <button id="btnGuardarMensagem">Guardar</button>
    <button id="btnCarregarMensagem">Carregar</button>
    <button id="btnRemoverMensagem">Remover</button>
    <p id="mensagemSalva"></p>

    <h2>Preferências do Utilizador (Objeto)</h2>
    <div id="preferenciasUsuario">
        <p>Tema: <span id="temaAtual"></span></p>
        <p>Notificações: <span id="notificacoesAtuais"></span></p>
        <button id="btnAlternarTema">Alternar Tema</button>
        <button id="btnAlternarNotificacoes">Alternar Notificações</button>
    </div>

    <script src="local_storage.js"></script>
</body>
</html>
```

E crie `local_storage.js` :

JavaScript

```
// local_storage.js

const inputMensagem = document.getElementById("inputMensagem");
const btnGuardarMensagem = document.getElementById("btnGuardarMensagem");
const btnCarregarMensagem = document.getElementById("btnCarregarMensagem");
const btnRemoverMensagem = document.getElementById("btnRemoverMensagem");
const mensagemSalva = document.getElementById("mensagemSalva");

const temaAtualSpan = document.getElementById("temaAtual");
const notificacoesAtuaisSpan = document.getElementById("notificacoesAtuais");
const btnAlternarTema = document.getElementById("btnAlternarTema");
const btnAlternarNotificacoes = document.getElementById("btnAlternarNotificacoe

const CHAVE_MENSAGEM = "minhaMensagem";
```

```
const CHAVE_PREFERENCIAS = "preferenciasUsuario";

// Função para atualizar a exibição das preferências
function atualizarPreferenciasDisplay() {
    const preferenciasString = localStorage.getItem(CHAVE_PREFERENCIAS);
    let preferencias = { tema: "claro", notificacoes: true }; // Valores padrão

    if (preferenciasString) {
        preferencias = JSON.parse(preferenciasString);
    }

    temaAtualSpan.textContent = preferencias.tema;
    notificacoesAtuaisSpan.textContent = preferencias.notificacoes ? "Ativadas"
}

// Carregar mensagem ao iniciar a página
window.addEventListener("load", () => {
    const mensagem = localStorage.getItem(CHAVE_MENSAGEM);
    if (mensagem) {
        mensagemSalva.textContent = `Mensagem carregada: "${mensagem}"`;
        inputMensagem.value = mensagem;
    } else {
        mensagemSalva.textContent = "Nenhuma mensagem guardada.";
    }
    atualizarPreferenciasDisplay();
});

btnGuardarMensagem.addEventListener("click", () => {
    const mensagem = inputMensagem.value;
    if (mensagem) {
        localStorage.setItem(CHAVE_MENSAGEM, mensagem);
        mensagemSalva.textContent = `Mensagem guardada: "${mensagem}"`;
    } else {
        mensagemSalva.textContent = "Por favor, digite uma mensagem para guarda
    }
});

btnCarregarMensagem.addEventListener("click", () => {
    const mensagem = localStorage.getItem(CHAVE_MENSAGEM);
    if (mensagem) {
        mensagemSalva.textContent = `Mensagem carregada: "${mensagem}"`;
        inputMensagem.value = mensagem;
    } else {
        mensagemSalva.textContent = "Nenhuma mensagem guardada.";
    }
});

btnRemoverMensagem.addEventListener("click", () => {
```

```
    localStorage.removeItem(CHAVE_MENSAGEM);
    mensagemSalva.textContent = "Mensagem removida.";
    inputMensagem.value = "";
});

btnAlternarTema.addEventListener("click", () => {
    const preferenciasString = localStorage.getItem(CHAVE_PREFERENCIAS);
    let preferencias = { tema: "claro", notificacoes: true };

    if (preferenciasString) {
        preferencias = JSON.parse(preferenciasString);
    }

    preferencias.tema = (preferencias.tema === "claro") ? "escuro" : "claro";
    localStorage.setItem(CHAVE_PREFERENCIAS, JSON.stringify(preferencias));
    atualizarPreferenciasDisplay();
});

btnAlternarNotificacoes.addEventListener("click", () => {
    const preferenciasString = localStorage.getItem(CHAVE_PREFERENCIAS);
    let preferencias = { tema: "claro", notificacoes: true };

    if (preferenciasString) {
        preferencias = JSON.parse(preferenciasString);
    }

    preferencias.notificacoes = !preferencias.notificacoes;
    localStorage.setItem(CHAVE_PREFERENCIAS, JSON.stringify(preferencias));
    atualizarPreferenciasDisplay();
});
```

Explicação do Exemplo

- Demonstra como usar `localStorage.setItem()`, `localStorage.getItem()` e `localStorage.removeItem()` para gerir dados simples (strings).
- Mostra como armazenar e recuperar objetos complexos (preferências do utilizador) usando `JSON.stringify()` para converter o objeto em string antes de guardar e `JSON.parse()` para converter a string de volta em objeto ao ler.
- A função `atualizarPreferenciasDisplay()` garante que a interface do utilizador reflete o estado atual das preferências armazenadas.
- O evento `window.addEventListener("load", ...)` é usado para carregar os dados do LocalStorage assim que a página é carregada.

Exercício do Dia 25

1. Crie um novo arquivo `lista_compras.html` e `lista_compras.js`.
2. No HTML, crie:
   - Um `input` de texto para adicionar itens à lista de compras.
   - Um botão "Adicionar Item".
   - Uma lista não ordenada (`<ul>`) vazia com um `id` para exibir os itens.
   - Um botão "Limpar Lista".
3. No JavaScript:
   - Ao carregar a página, verifique se há uma lista de compras guardada no LocalStorage. Se houver, carregue-a e exiba os itens na `<ul>`.
   - Quando o botão "Adicionar Item" for clicado:
     - Adicione o item do input à lista (pode ser um array de strings).
     - Guarde a lista atualizada no LocalStorage (lembre-se de usar `JSON.stringify()`).
     - Atualize a exibição da lista na página.
   - Quando o botão "Limpar Lista" for clicado:
     - Remova a lista do LocalStorage.
     - Limpe a exibição da lista na página.

Dia 26-28: Projeto Final: Aplicação de Tarefas (To-Do List)

Conceitos

Estes três dias são dedicados à construção de um projeto final que integra todos os conhecimentos adquiridos nas últimas semanas: HTML para estrutura, CSS para estilo e responsividade, e JavaScript para interatividade, manipulação do DOM, eventos e armazenamento local.

O objetivo é criar uma **Aplicação de Tarefas (To-Do List)** completa, onde o utilizador pode:

- Adicionar novas tarefas.
- Marcar tarefas como concluídas.
- Remover tarefas.
- Filtrar tarefas (todas, ativas, concluídas).
- Ter as tarefas persistidas no navegador (usando LocalStorage).

Requisitos Técnicos

- **HTML:** Estrutura clara para o input de tarefas, a lista de tarefas e os botões de filtro.
- **CSS:** Estilo moderno e responsivo. As tarefas concluídas devem ter um estilo diferente (ex: riscado).
- **JavaScript:**
  - **Manipulação do DOM:** Adicionar, remover e atualizar elementos da lista de tarefas dinamicamente.
  - **Event Listeners:** Reagir a cliques nos botões (adicionar, marcar como concluída, remover, filtrar).
  - **LocalStorage:** Guardar e carregar as tarefas para que persistam entre sessões do navegador.
  - **Lógica de Programação:** Funções para gerir as tarefas (adicionar, remover, alternar estado), filtros e renderização da lista.

Estrutura Sugerida (JavaScript)

1. **Array de Tarefas:** Mantenha todas as tarefas num array de objetos JavaScript. Cada objeto de tarefa pode ter `id`, `descricao` e `concluida`.
2. **Função `renderizarTarefas()`:** Esta função deve ser responsável por limpar a lista HTML existente e recriá-la com base no array de tarefas atual. Deve ser chamada sempre que o array de tarefas for modificado.
3. **Funções de Manipulação:**
   - `adicionarTarefa(descricao)`: Cria um novo objeto de tarefa, adiciona-o ao array e chama `renderizarTarefas()`.
   - `alternarConcluida(id)`: Encontra a tarefa pelo `id` e inverte o seu estado `concluida`. Chama `renderizarTarefas()`.
   - `removerTarefa(id)`: Remove a tarefa do array pelo `id`. Chama `renderizarTarefas()`.
4. **Funções de Filtro:** Modificam o array de tarefas a ser exibido ou a lógica de `renderizarTarefas()` para mostrar apenas as tarefas relevantes.
5. **LocalStorage:** Sempre que o array de tarefas for modificado, guarde-o no LocalStorage. Ao carregar a página, tente carregar as tarefas do LocalStorage.

Exercício do Dia 26-28

Crie os arquivos `todo_list.html`, `todo_list.css` e `todo_list.js`.

Dia 26: Estrutura e Adicionar Tarefas

1. Crie a estrutura HTML básica: um título, um input para a tarefa, um botão "Adicionar", e uma `<ul>` vazia.

2. No JavaScript, implemente a funcionalidade de adicionar tarefas. Ao clicar no botão, a tarefa deve ser adicionada a um array e exibida na lista HTML.
3. Adicione um estilo básico para a lista e os itens.

Dia 27: Marcar como Concluída e Remover

1. Modifique a função `renderizarTarefas()` para que cada item da lista tenha:
   - Um `checkbox` para marcar/desmarcar como concluída.
   - Um botão "Remover" (ou um ícone de lixo).
2. Implemente os `event listeners` para o `checkbox` (alternar estado `concluida`) e para o botão "Remover".
3. Atualize o CSS para que as tarefas concluídas tenham um estilo diferente (ex: `text-decoration: line-through;`).

Dia 28: Persistência e Filtros

1. Implemente o uso do LocalStorage para guardar e carregar as tarefas. As tarefas devem persistir mesmo após fechar e reabrir o navegador.
2. Adicione botões de filtro (ex: "Todas", "Ativas", "Concluídas").
3. Implemente a lógica para que, ao clicar num botão de filtro, apenas as tarefas correspondentes sejam exibidas na lista.

Dica: Comece pequeno e adicione funcionalidades gradualmente. Teste cada parte à medida que a constrói.

Dia 29: Git, GitHub e Deploy (Netlify/Vercel)

Conceitos

Git é um sistema de controlo de versão distribuído que rastreia as alterações no código-fonte durante o desenvolvimento de software. Ele permite que vários desenvolvedores trabalhem no mesmo projeto sem conflitos e mantém um histórico completo de todas as alterações.

Comandos Git básicos:

- `git init`: Inicializa um novo repositório Git.
- `git add .`: Adiciona todos os arquivos modificados à área de staging.
- `git commit -m "Mensagem"`: Guarda as alterações no histórico do repositório.
- `git status`: Mostra o estado atual do repositório.
- `git log`: Mostra o histórico de commits.

- `git branch` : Lista, cria ou elimina branches.
- `git checkout nome_da_branch` : Muda para uma branch existente.
- `git merge nome_da_branch` : Integra as alterações de uma branch noutra.

GitHub é uma plataforma de hospedagem de código-fonte baseada em Git. Ele fornece controlo de versão, colaboração e funcionalidades de gestão de projetos. É onde a maioria dos projetos de código aberto são hospedados e é essencial para qualquer desenvolvedor.

Deploy é o processo de publicar a sua aplicação web para que ela possa ser acedida publicamente na internet. Plataformas como **Netlify** e **Vercel** tornam o deploy de sites estáticos (HTML, CSS, JavaScript) extremamente fácil e gratuito.

Passos para Deploy com GitHub Pages (Alternativa Simples)

1. **Crie um Repositório no GitHub:** Vá ao GitHub, faça login e crie um novo repositório. Dê-lhe um nome significativo (ex: `meu-primeiro-site`).
2. **Inicialize o Git Localmente:** No terminal, navegue até a pasta do seu projeto e execute:

   Bash
   ```bash
   git init
   git add .
   git commit -m "Primeiro commit do projeto"
   ```

3. **Conecte ao GitHub:** Adicione o repositório remoto e faça o primeiro push:

   Bash
   ```bash
   git remote add origin https://github.com/SEU_USUARIO/SEU_REPOSITORIO.git
   git branch -M main
   git push -u origin main
   ```

4. **Ative o GitHub Pages:** No seu repositório no GitHub, vá em `Settings` -> `Pages` . Selecione a branch `main` (ou `master`) e a pasta `/(root)` como fonte. Salve. O seu site estará disponível num URL como `https://SEU_USUARIO.github.io/SEU_REPOSITORIO/` em alguns minutos.

Passos para Deploy com Netlify/Vercel (Recomendado para Projetos Maiores)

1. **Crie um Repositório no GitHub:** Como acima.
2. **Faça o Push do seu Código:** Certifique-se de que todo o seu código está no GitHub.
3. **Conecte ao Netlify/Vercel:**

- Vá ao Netlify ou Vercel e faça login (pode usar o GitHub).
- Clique em "Add new site" ou "New Project".
- Selecione "Import an existing project from a Git repository" e escolha o seu repositório do GitHub.
- As configurações de build geralmente são detetadas automaticamente para sites estáticos. Clique em "Deploy site" ou "Deploy".

4. **Site Online:** O seu site será automaticamente publicado e receberá um URL único. Qualquer push futuro para a branch principal do seu repositório no GitHub acionará um novo deploy automático.

Exercício do Dia 29

1. Escolha um dos projetos que criou esta semana (ex: a Calculadora de Impostos ou a Aplicação de Tarefas).
2. Inicialize um repositório Git na pasta do projeto.
3. Faça commits regulares à medida que trabalha.
4. Crie um novo repositório no GitHub e faça o push do seu projeto para lá.
5. Faça o deploy do seu projeto usando o GitHub Pages, Netlify ou Vercel. Partilhe o link com um amigo ou familiar!

Dia 30: Revisão, Portfólio e Próximos Passos

Conceitos

Parabéns! Chegou ao fim do seu plano de 30 dias. Este dia é crucial para consolidar o que aprendeu, planear o futuro e começar a construir o seu portfólio.

Revisão:

- Revise todos os conceitos de HTML, CSS e JavaScript. Refaça os exercícios que achou mais desafiadores.
- Entenda a interconexão entre as três tecnologias: HTML para estrutura, CSS para estilo e JavaScript para interatividade.
- Pense em como os projetos que construiu poderiam ser melhorados ou expandidos.

Portfólio:

Um portfólio é a sua montra como desenvolvedor. É essencial para mostrar as suas habilidades a potenciais empregadores ou clientes. Os projetos que construiu durante este mês são um excelente começo.

- **Escolha os Melhores Projetos:** Selecione os 2-3 projetos que melhor demonstram as suas habilidades.
- **Limpe o Código:** Certifique-se de que o código está bem organizado, comentado e sem erros.
- **Escreva uma Boa Documentação:** Para cada projeto, inclua um `README.md` no GitHub que explique o que o projeto faz, as tecnologias usadas e como executá-lo.
- **Crie um Site de Portfólio:** Pode ser um site simples (usando o que aprendeu!) que lista os seus projetos com links para os repositórios do GitHub e para os demos online (Netlify/Vercel).

Próximos Passos:

O desenvolvimento web é um campo vasto e em constante evolução. Após esta base sólida, aqui estão algumas direções para continuar a sua aprendizagem:

- **Aprofundar JavaScript:** Event Loop, assincronismo (Promises, Async/Await), módulos ES6, programação orientada a objetos.
- **Frameworks/Bibliotecas JavaScript:**
  - **React:** Uma biblioteca popular para construir interfaces de utilizador interativas.
  - **Vue.js:** Um framework progressivo para construir interfaces de utilizador.
  - **Angular:** Um framework completo para construir aplicações web complexas.
- **Ferramentas de Build:** Webpack, Vite (para otimizar e empacotar o seu código).
- **Pré-processadores CSS:** Sass/Less (para escrever CSS mais poderoso e organizado).
- **Backend Development:** Node.js (com Express), Python (com Django/Flask), Ruby (com Rails), PHP (com Laravel) para construir a parte do servidor das aplicações.
- **Bases de Dados:** SQL (PostgreSQL, MySQL), NoSQL (MongoDB).
- **TypeScript:** Um superset de JavaScript que adiciona tipagem estática, melhorando a robustez do código.
- **Testes:** Aprender a escrever testes para o seu código.

Exercício do Dia 30

1. **Revisão:** Passe algumas horas a rever os seus projetos e o código que escreveu. Tente identificar áreas onde poderia ter feito melhor ou onde ainda tem dúvidas.
2. **Melhore um Projeto:** Escolha um dos seus projetos e adicione uma pequena funcionalidade extra ou melhore o seu estilo/responsividade.
3. **Plano de Portfólio:** Comece a esboçar como seria o seu site de portfólio. Quais projetos incluiria? Que informações sobre si gostaria de partilhar?

4. **Pesquise:** Dedique algum tempo a pesquisar sobre React, Vue ou Node.js. Veja tutoriais introdutórios para ter uma ideia do que são e qual deles mais lhe interessa para o próximo passo da sua jornada.

Lembre-se: Este é apenas o começo. A chave para se tornar um bom desenvolvedor é a aprendizagem contínua e a prática constante. Continue a construir, a experimentar e a aprender!

Referências

[1] MDN Web Docs. Learn web development. Disponível em:

[2] W3Schools. HTML Tutorial. Disponível em:

[3] freeCodeCamp. Learn to Code — For Free — forever. Disponível em:

[4] The Odin Project. Learn Web Development for Free. Disponível em:

[5] CSS-Tricks. A Complete Guide to Flexbox. Disponível em:

[6] CSS-Tricks. A Complete Guide to CSS Grid. Disponível em:

[7] Google Developers. Responsive Web Design Basics. Disponível em:

[8] Web Content Accessibility Guidelines (WCAG). Overview. Disponível em:

[9] Google Search Central. Search Engine Optimization (SEO) Starter Guide. Disponível em:

[10] Git. Documentation. Disponível em:

[11] GitHub. About GitHub. Disponível em:

[12] Netlify. Build, deploy, and scale modern web projects. Disponível em:

[13] Vercel. Develop. Preview. Ship. Disponível em: